Final Project – Notes

Purpose

These notes present the final project requirements as detailed in the Coursera course ***Developing Front-End Apps with React***. The final project application is called ***Paradise Nursery***.

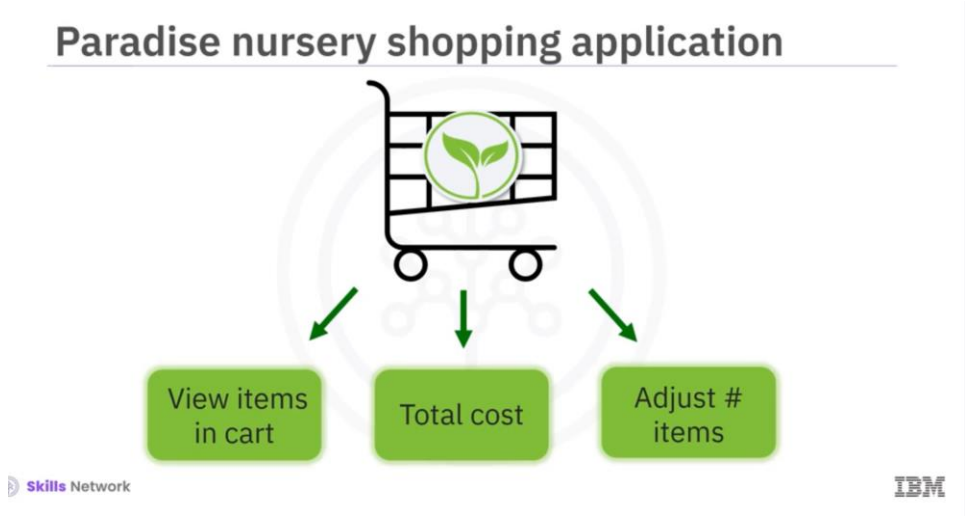
In addition, this document contains, my notes analyzing what was provided, and all changes made in order to implement the requirements.

I present all the material provided by the course in these notes – so there will be some duplication.

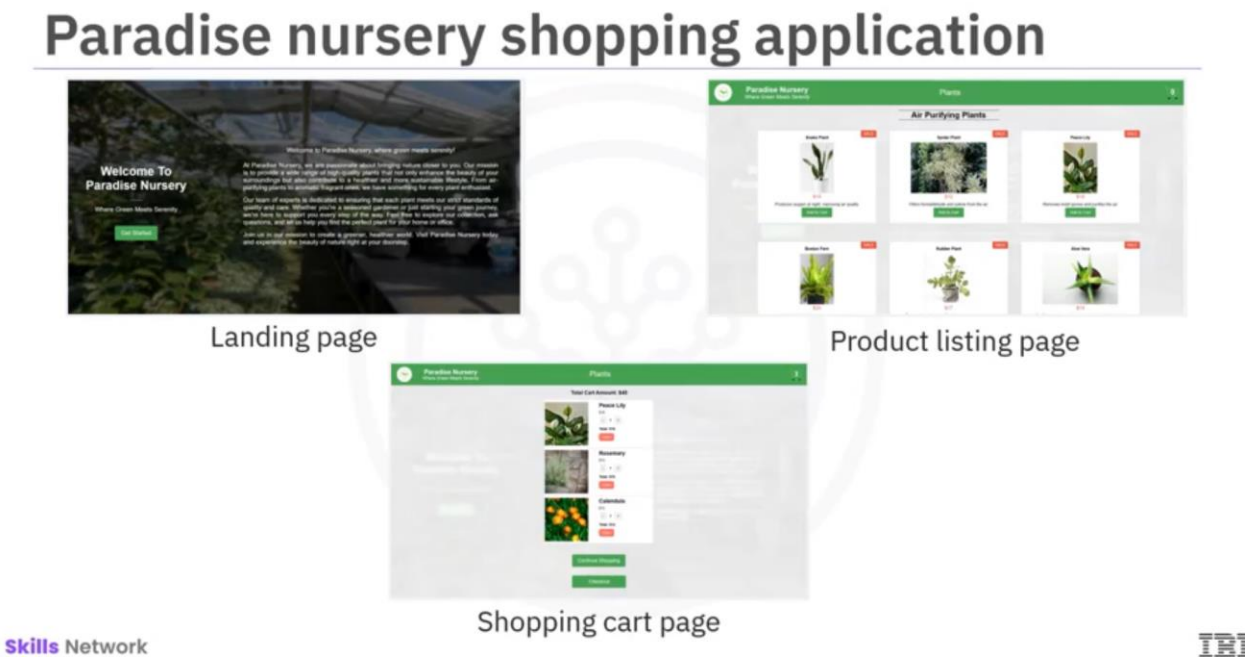
Requirements

Welcome to the React Final Project Walkthrough. For your final project, you will create the front end of a paradise nursery shopping application.

The application interface will allow you to browse a variety of house plants and add them to a shopping cart.



The shopping cart feature lets you see all your items and their total cost. Your shopping cart should also allow you to adjust the number of items in the cart before checkout.

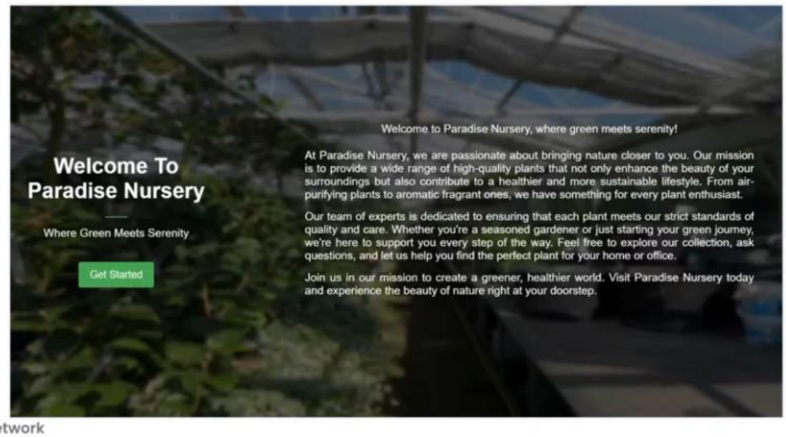


Your application will have three pages, **a landing page**, **a product listing page**, and a **shopping cart page**. Let's preview the application and tour the user experience.

Note: The source code provided refers to an “About Us” page in the code comments[1]. This is probably now the “Landing Page”

[1] This is why some folks hate code comments – they become outdated and never in sync with the latest code.

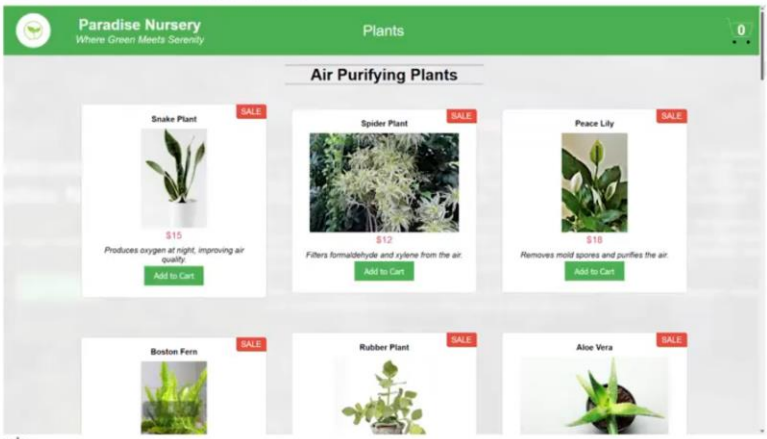
Landing page



Here, you see a sample landing page for the Paradise Nursery shopping application. Create a landing page similar to this one that engages your customers with your product line and provides entry into the main application. Elements on this page should include a paragraph about your company, a background image, your company name, and a Get Started button linking to the product listing page.

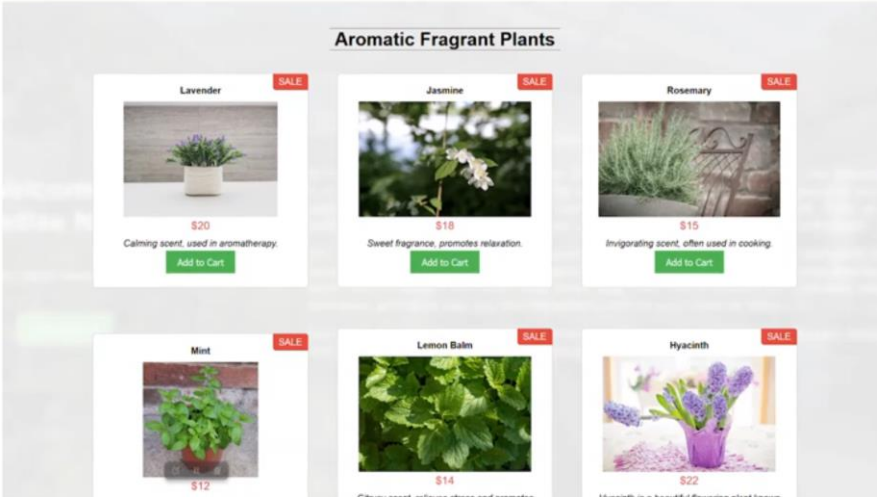
Note: The Landing page content is provided in the lab starting code.

Product listing page



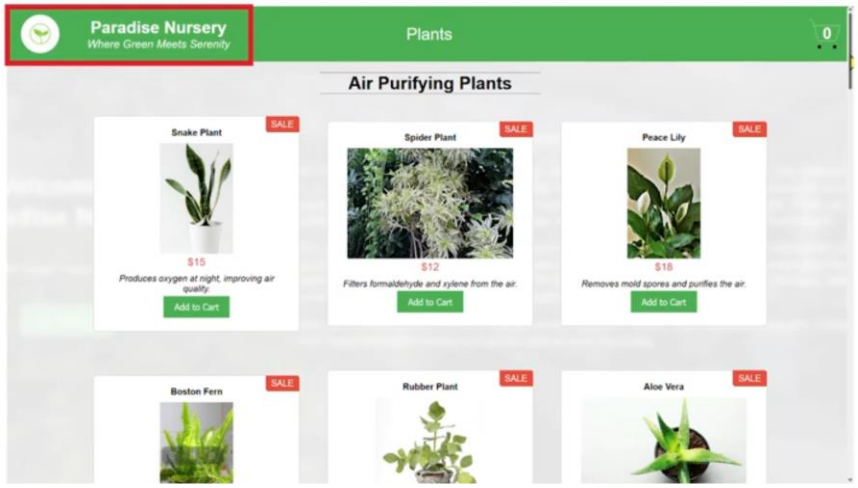
After selecting Get Started, the application should direct you to a product listing page like the one shown here. This page allows you to browse the house plants in your product line. Page elements include a page header and a card for each plant. The page should group the plants by a common feature, such as air purifying or aromatic.

Product listing page

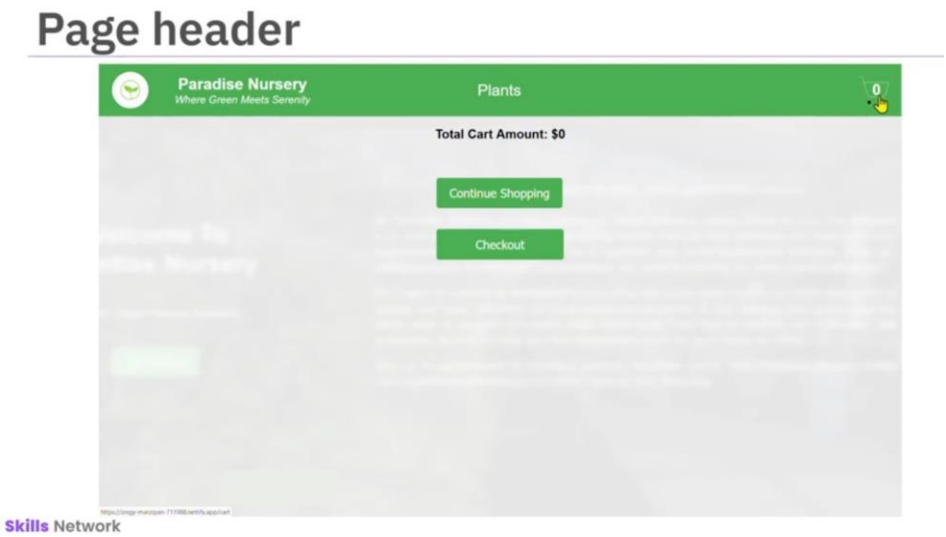


Some sections may have a single plant in multiple groups.

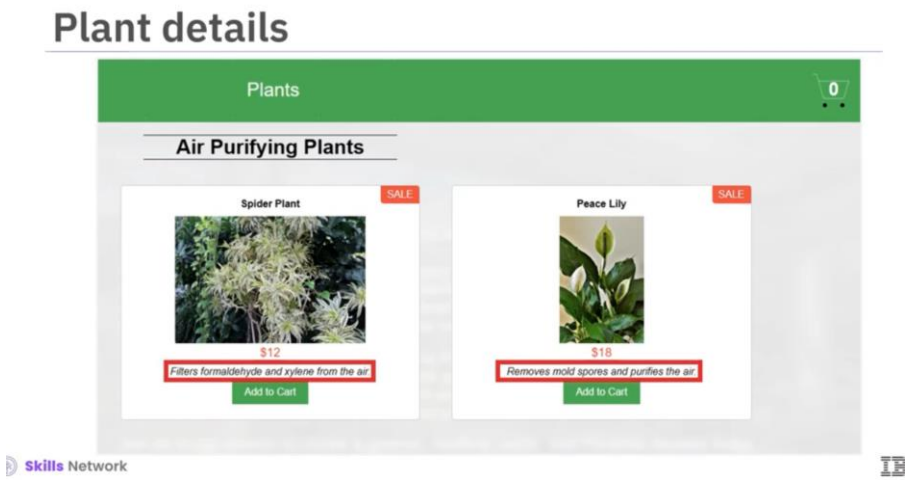
Page header



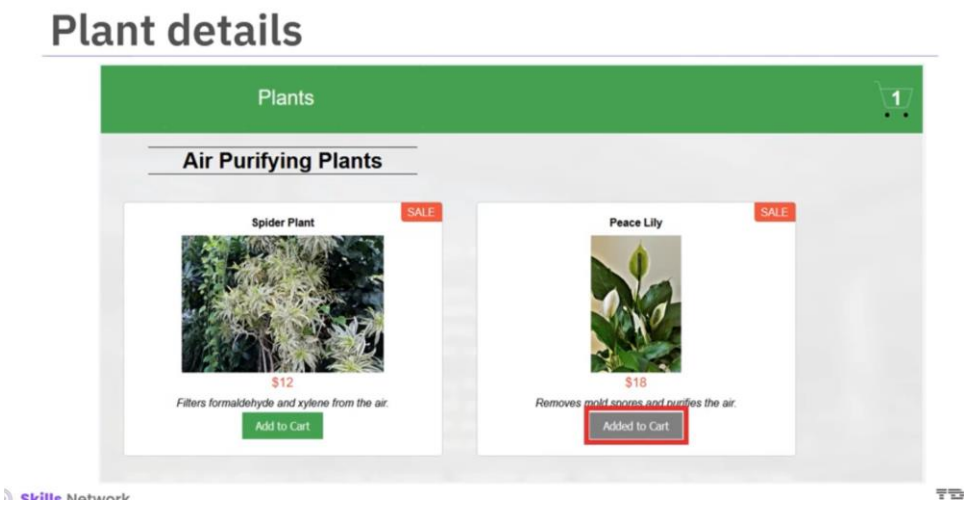
The page header should include the company name and logo, which will navigate back to the landing page. It should also include a tagline and a shopping cart icon, displaying the current number of items. The application will navigate to the shopping cart page if you select the cart icon.



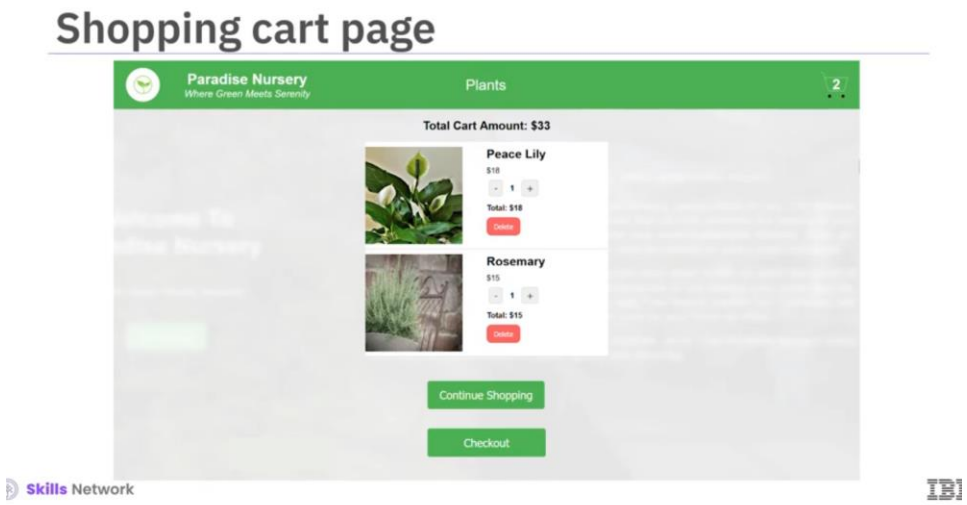
Each plant should have its own card, similar to what you see here. Each card has a thumbnail, the plant name, its price, a brief description, and an option to add the plant to the cart with the label Add to Cart.



If you add the item to the cart, disable this option and change the label to Added to Cart.

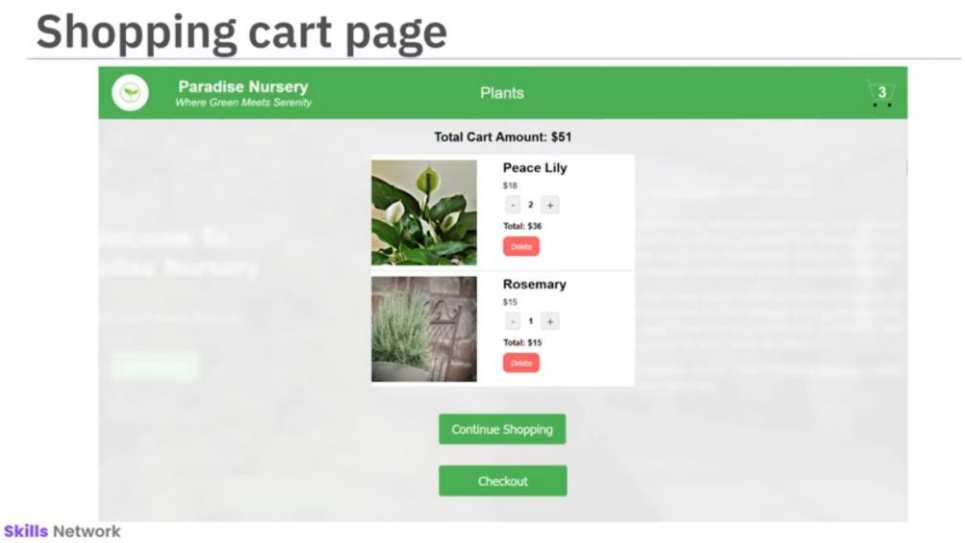


Once you select the Add to Cart option, the number on the cart icon should increase by one.

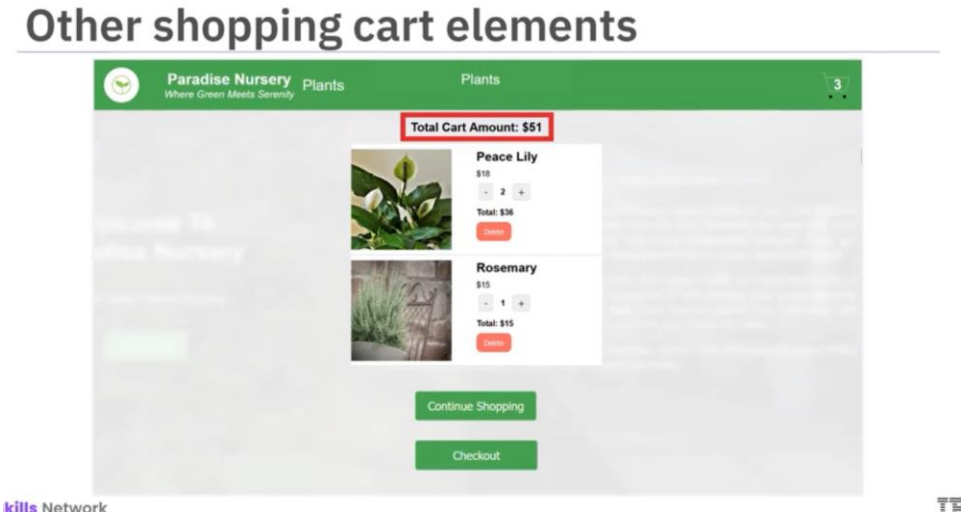


Now, take a look at the shopping cart page and its components.

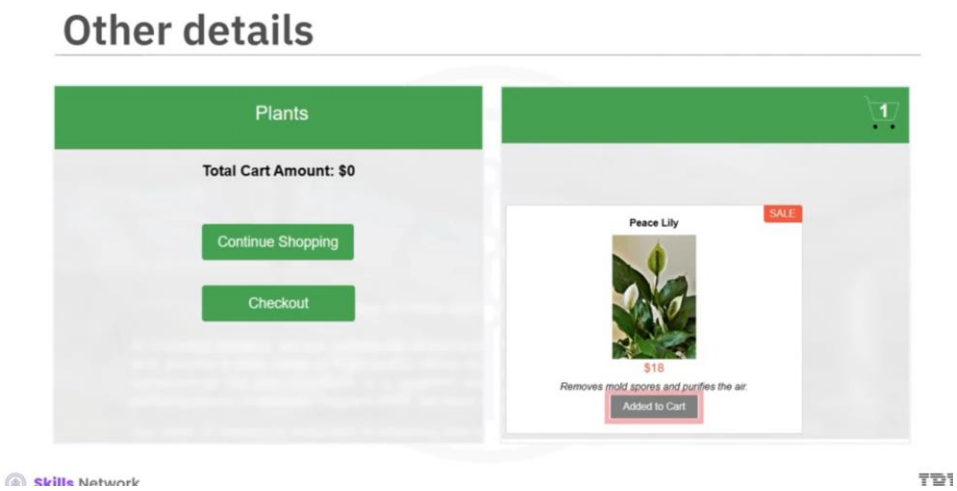
Note: The starting code provided in the lab displays the content of the cart as shown above. You primary job will be to determine the totals, finish CartSlice code to perform count updates, remove items.



You should organize the shopping cart using cards, one for each type of plant added to the cart from the product listing page. Each card should display a thumbnail image, a delete option, and the unit price. You can increase or decrease the number of items using an increase or decrease button, which, when selected, increments or decrements the number on the card. You will also need to update the total number of items in the cart. Also, the card should display the subtotal for the number of that plant type as shown here.

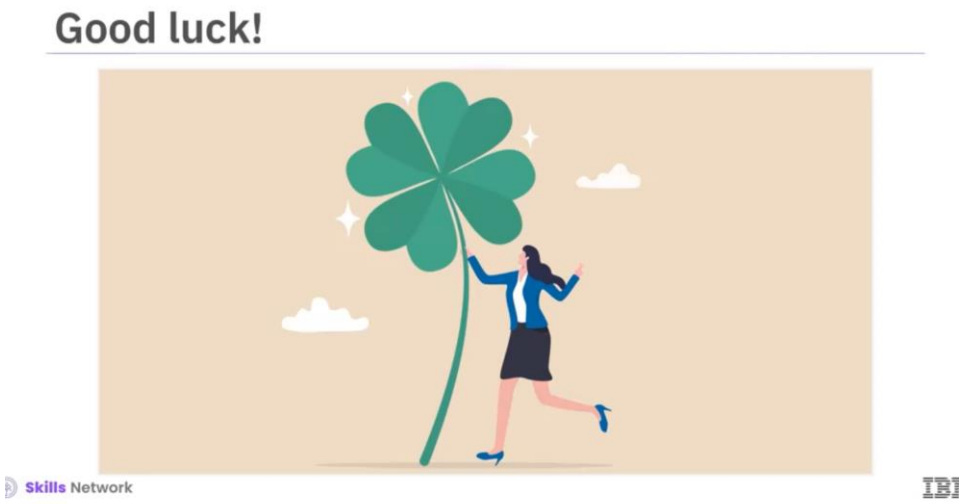


Next, consider the other elements on the shopping cart page. Prominently display the total number of plants in the cart, the sum total costs of all items, and a Checkout button. Use the same page header from the product listing page.



You will need to handle the consequences of user events, such as when the number of items in the cart decreases to zero. This affects other elements, such as the disabled Added to Cart option on the product listing page. You need to re-enable that option to select it again when you return to the product listing page. Also, remember to adjust the number of items shown on the cart icon in the header when using the increase and decrease buttons.

We encourage you to complete the practice project before developing this application. Much of the functionality and code you wrote for that project will transfer to this one. After completing these projects, you will feel comfortable applying many of the ideas you learned throughout the course to a single application. Good luck!



Reading: Final Project Overview

For your final project, you will create the front end of a "Paradise Nursery" shopping application. The application interface will allow you to browse a variety of houseplants and add them to a shopping cart. The shopping cart feature lets you see all your items and their total cost. Your shopping cart should also allow you to adjust the number of items in the cart before checkout.

Completing the practice project will help you with the final project. We designed it so that many of the components, code, and skills for that project can be transferred to this final project with some updates.

Your application should have three pages: a landing page, a product listing page, and a shopping cart page. See the lab for details on the specific functionality requirements on each page. The product walkthrough video also demonstrates how your completed project should look and behave.

Landing page

The landing page should have the following elements:

- A background image
- A paragraph about the company
- The company name
- A **Get Started** button linking to the product page

The product listing page should have at least six houseplants for sale, organized into three or more categories. Each plant should have the following details about it:

- A thumbnail image
- Plant name
- Price
- An **Add to Cart** button

Product listing page

The product listing and shopping cart pages should have a header. The header should have a shopping cart icon with a number that updates dynamically, displaying the total number of items in the cart. The header should also contain navigation to either of the other pages, depending on which page you are on.

Shopping cart page

The shopping cart page displays all the details about the items in it. It should prominently display the total number of plants in the cart, the total cost of all items in the cart, a continue shopping button, and a checkout button.

On this page, you should be able to see each type of plant in the shopping cart, including its thumbnail and name. You should also display its unit price and the total of all the items in the cart. You should provide an increase or decrease button to change the number of that plant type in the cart, which also updates the shopping cart icon in the header and the total number of items in the cart. Finally, you should also include a delete button for each type of plant in the cart.

You must store your project code in a public GitHub repository. Also, you will use GitHub Pages to deploy your app.

Evaluation Criteria – 50 Points

You can submit your project deliverables in one of the following ways:

Option 1: AI-Graded Submission and Evaluation
When you choose Option 1, you will be redirected to an AI tool where you can upload your deliverables, which may include URLs, terminal outputs, code snippets, or screenshots. You will then receive an AI-generated grade that will automatically reflect on your Coursera progress page.

Option 2: Peer-Graded Submission and Evaluation
When you choose Option 2, you will upload your deliverables—such as URLs, terminal outputs, code snippets, or screenshots—through the My Submission section. Your submission will then be reviewed either by your peers or by Coursera's AI grader.

We recommend using Option 1 for faster grading. However, if you face any issues or cannot access it, you may use Option 2 instead.

If you encounter any grading problems, please reach out to the Course Team through the Discussion Forums.

Please find the details of the Grading Criteria below:

Criteria for Option 1: AI-Graded Submission and Evaluation:

Task 1:

Submit the public GitHub URL of the **README.md** file containing the project name details.(2 points)

Task 2:

Submit the public GitHub URL of **AboutUs.jsx**, which includes details about the company.(1 point)

Task 3:

Submit the GitHub URL of **App.css**, which includes the code implementing the background image for the Paradise Nursery landing page.(1 point)

Task 4:

Submit the public GitHub URL of **App.jsx**, which includes the Paradise Nursery landing page with the company name and a **"Get Started"** button.(3 points)

Task 5:

Submit the public GitHub URL of **CartSlice.jsx**, which includes the code implementing the Redux slice for the shopping cart.(4 points)

Task 6:

Submit the public GitHub URL of **ProductList.jsx**.(16 points)
Your implementation must include the following functionalities:

- Display at least six unique houseplants per category, each with a thumbnail, name, and price. (2)
- Group the plants into at least three categories.(2)
- Include "Add to Cart" buttons for each plant that:
    - Add the product to the shopping cart (2)
    - Disable the button after the product is added (2)
    - Update and display the shopping cart quantity (2)
- Include a navbar that appears on both Product Listing and Cart pages (2)
- Navbar provides navigation links to all three pages: Home, Plants, and Cart (2)
- Display a cart icon showing the total number of items dynamically(2)

Task 7:

Submit the public GitHub URL of **CartItem.jsx**, displaying the Shopping Cart page.(23 points)
Your implementation must include the following functionalities:

- Show the total cart amount of the plants(2)
- Show the total cost for each plant in the cart (2)
- Each plant in the cart should display a thumbnail, name, and unit price (6)
- Include button to increase the quantity, updating the cart correctly (4)
- Include button to decrease the quantity, updating the cart correctly (4)
- Include a delete button for each item to remove it from the cart (2)
- Include a checkout button that shows "Coming Soon" or a similar message (1)
- Include a continue shopping button that links back to the product listing page (2)

Criteria for Option 2: Peer-Graded Submission and Evaluation:

GitHub Link (6 points)

- GitHub repository public URL: 2 points
- Redux-related files and code: 4 points

Deployed app (44 points)

Landing Page (5 points)

- Background image: 1 point
- Paragraph about the company: 1 point
- Company name: 1 point
- Get Started button linking to the product listing page: 2 points

Product Listing Page (9 points)

- Six unique houseplants for sale, each displaying thumbnail, name, and price: 2 points
- Group the plants into at least three categories on the page: 1 point
- An Add to Cart button for each plant, with the following behavior: 6 points
    - After selecting it, the shopping cart icon increases by one.
    - After selecting it, the button becomes disabled.
    - After selecting it, the appropriate plant gets added to the shopping cart.

Header (7 points)

- Displays on both the product listing page and shopping cart page: 2 points
- A shopping cart icon with a value that displays the total number of items in the cart: 3 points
- Navigation to either of the other pages: 2 points

Shopping Cart Page (23 points)

- The total number of plants in the cart: 2 points
- The total cost of all items in the cart: 2 points
- Each plant type in the cart displays a thumbnail, name, and unit price: 6 points
- Increase button for each plant type in the cart: 4 points
    - Increments the number of items in the cart by one each time it`s clicked and updates all appropriate values.
- Decrease button for each plant type in the cart: 4 points
- Decrements the number of items in the cart by one each time it`s clicked and updates all appropriate values.
- A delete button: 2 points
- A checkout button (displays the message "Coming Soon" or similar):1 point
- A continue shopping button that links to the product listing page: 2 points

Final Project: Paradise Nursery Shopping Application

In this final project, you will create a shopping cart application for an online plant shop that offers a variety of house plants.

To launch the lab, check the box below indicating "I agree to use this app responsibly.", and then click on the "Launch App" button. This will open up the lab environment in a new browser tab.

This lab uses IBM Skills Network Labs (SN Labs), which is a virtual lab environment used in this course. Upon clicking "Launch App" your Username and Email will be passed to Skills Network

Labs and will only be used for communicating important information to enhance your learning experience, in accordance with IBM Skills Network Privacy policy.

In case you need to download or print the lab instructions, click HERE to open the lab instructions in a new window.

This course uses a third-party app, Final Project: Paradise Nursery Shopping Application, to enhance your learning experience. The app will reference basic information like your name, email, and Coursera ID.

Lab Notes

Final Project: Paradise Nursery Shopping Application

Estimated time needed: 1 hour

Introduction

In this final project, you will create a shopping cart application for an online plant shop which offers a variety of house plants.

The *Paradise Nursery* shopping cart features will include:

- A Landing page with a button linking to the product listing page
- A navigation bar with links to the landing, product listing, and shopping cart pages
- A card for each plant that showcases the different plants along with their images, name, description, cost and an **Add to cart** button.
- A minimum of two sections describing the plants in that section. For example, "Aromatic Plants" and "Medicinal Plants".
- A cart page which displays the products in the cart.
- The cart should have a card for each type of plant in the cart. Each card should have the thumbnail, the unit cost, the cost for all of the plants of that type and buttons to increase and decrease the quantity along with **Delete** button.
- A **Continue Shopping** and **Checkout** buttons

You will apply the knowledge and skills gained from the practice project to implement dynamic functionalities in the final project. Hence it is recommended to complete the practice project before attempting the final project. For example, features like displaying the cart quantity in the navbar icon and updating the total cost when the user changes the item quantity will be part of the final project.

Note:

1. Please ensure that all you files are updated and pushed to the public github repository and your deployed URL is live and publically accessible.
2. As mentioned in the *"Final Project Overview"*, you can submit your project deliverables through either Option 1: AI-Graded Submission and Evaluation or Option 2: Peer-Graded Submission and Evaluation.
3. **Option 1: AI-Graded Submission and Evaluation**
   Submit the **public GitHub repository URLs** for the following project files: README.md, AboutUs.jsx, App.css, App.jsx, CartSlice.jsx, ProductList.jsx, and CartItem.jsx. Your project code must be stored in a **public GitHub repository named e-plantShopping**.
4. **Option 2: Peer-Graded Submission and Evaluation**
   Submit the **deployed application URL** and the **public GitHub repository URL** for peer evaluation.When you deploy, make a note of the **live application URL** Please ensure-

- All project files must be **committed and pushed** before submission.
- Your application must be **deployed** and the **live URL** should be available.
- You may host your app using **GitHub Pages** or any other hosting platform of your choice.

Learning objectives

After completing this lab, you will be able to:

- React Components: Create functional React components using component composition and nesting.
- State Management with Hooks: Implement React Hooks, specifically the useState and useEffect hooks. You will manage component-level state using hooks to control the visibility of elements.
- Redux Integration: Integrate Redux within an application using Redux concepts like actions, reducers, and the store.

- Rendering Dynamic Data: Dynamically render data fetched from an array of objects into the UI. You will map over arrays to generate lists of components.
- Handling Events and Conditional Rendering: Handle user events such as button selection and trigger corresponding actions.

Prerequisites

- Basic knowledge GitHub and your own GitHub account
- Understanding of React function components, props, hooks and React Redux Toolkit
- Web browser with a console (such as Chrome DevTools or Firefox Console)
- It is mandatory to complete the Practice Project

Development Tasks

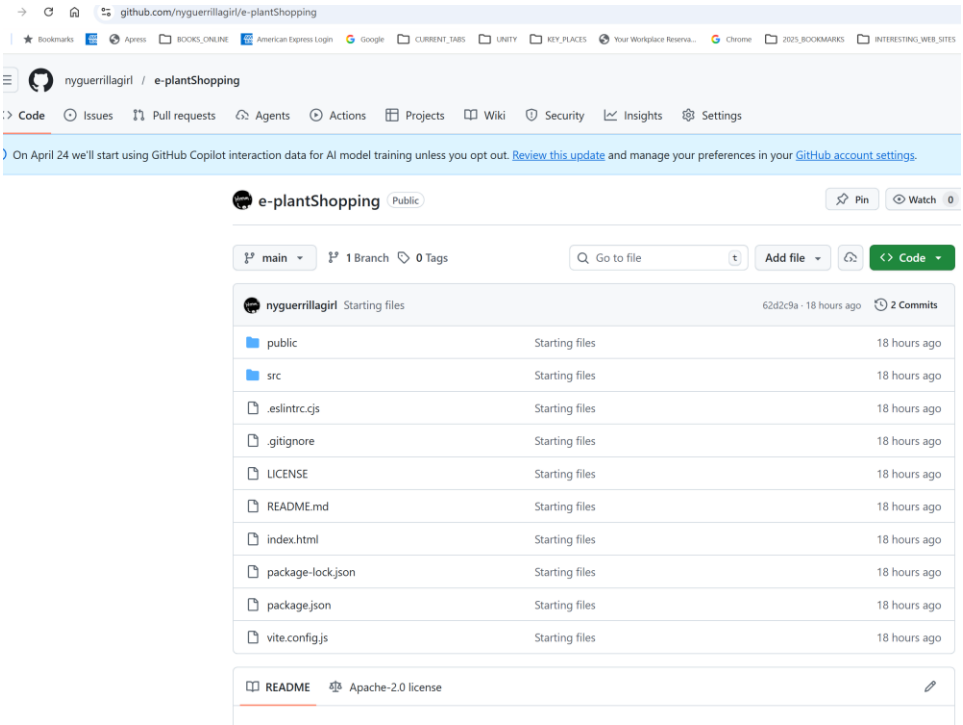
I was a bit surprised to discover that if you opened the lab for this final project it would pretty much tell you everything (code and all) you need to do to complete this final project. I decided to go step by step to ensure I understood everything.

Task 1: Getting the files and building the initial project provided

I like working locally on my own desktop. So I did the following to move the lab files into my own github repo:

1. Created a github repo on my own account: https://github.com/nyguerrillagirl/e-plantShopping. I only put in a blank README.md file.
2. I then created a final_project folder on my machine and checked out my github repo (which is mostly empty), entered: git clone https://github.com/nyguerrillagirl/e-plantShopping
3. Navigated to a temp folder on my local machine to clone the final project in there
4. Enter: git clone https://github.com/ibm-developer-skills-network/e-plantShopping.git
5. cd to the folder e-plantShopping.git
6. I removed the .git folder and copied the other files into my final_project/e-plantShopping repo.
7. I then updated my github repo:
    a. `git add .`
    b. `git commit -m “Starting files”`
    c. `git push origin main`

If you examine your repo on github you will see:



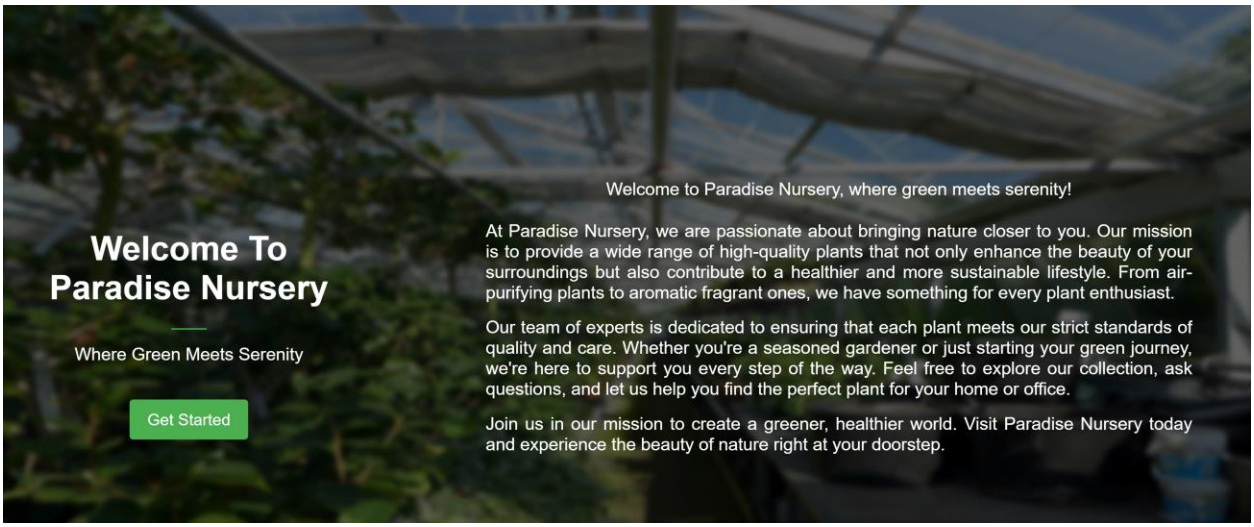
Task 2: Start the application (as is)

The first thing I did was start the project as is to see what is working.

- Open the project in Visual Studio Code
- Open a new terminal
- Enter: npm run dev

```
VITE v5.2.10  ready in 149 ms

➜ Local:   http://localhost:5173/shoppingreact
➜ Network: use --host to expose
➜ press h + enter to show help
```

- Enter the url provided into a browser (or simply CTRL+Click)



The landing page appears. If you click on “Get Started” you see an empty “Product Listing” page.

Task 3: Review the files provided

main.jsx

This is the main file where our application kicks off.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
import { Provider } from 'react-redux'
import store from './store.js'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
    <App />
    </Provider>
  </React.StrictMode>,
)
```

It has the typical things we expect to see when an application is using Redux. The import of `Provider` and the import of our `store`.

`Provider` is a special component from ***react-redux*** whose job is to make our redux store available to every component in our React application. Without it, hooks like:

- `useSelector()`
- `useDispatch()`

would not work, because React component would have no idea that a Redux store even exists. We can think of it as “plugging in” Redux to React.

Once the store is provided:

- useSelector() can read values from the Redux state
- useDispatch() can send actions to the store
- Components automatically re-render when the parts of the state they use change.

All this works ***without manually passing props***.

App.jsx

This component handles showing the landing page elements or the product list page elements via <ProductList>

The component uses:

```
  const [showProductList, setShowProductList] = useState(false);
```

to handle which state we are in. When `showProductList` is `false` (the default) we show the landing page information (<AboutUs/>), otherwise we show the <ProductList />. The code:

```
      <div className={`product-list-container ${showProductList ? 'visible' :
''}`}>
        <ProductList onHomeClick={handleHomeClick}/>
      </div>
```

. . . is doing three things at once.

1. It renders a `<div>` that wraps the component `<ProductList>` . The `ProductList` component receives one prop – onHomeClick function.
2. It conditionally add a CSS class based on state. This part:

```
${showProductList ? 'visible' : ''}
```

Means:

- If showProductList is true, that class visible is added
- If showProductList is false, nothing is added

So the final className becomes either:

```
1. "product-list-container visible"
```

Or

```
1. "product-list-container"
```

This is a very common React pattern for toggling visibility, animations, or transitions.

3. It allows CSS to control how the product list appears. The css in App.css is:

```
/* ProductList.css */

.product-list-container {
  position: absolute;
  overflow-x: hidden;
  top: 100vh;
  /* Initially off-screen */
  left: 0;
  width: 100%;
  height: 100vh;
  background-color: rgba(255, 255, 255, 0.9);
  /* Adjust background opacity as needed */
  transition: top 0.5s ease;
  /* Smooth transition for top property */
  overflow-y: auto;
  /* Enable vertical scrolling */
  z-index: 2;
  /* Ensure it's above landing page content */
  backdrop-filter: blur(5px);
}

.product-list-container.visible {
  top: 0;
}
```

The above essentially builds a ***slide-up overlay panel*** that starts off-screen and smoothly moves into view when the `.visible` class is added.

The container:

- Starts ***below the screen*** (`top: 100vh`)
- Slides ***up into view*** when `.visible` sets `top: 0`
- Covers the whole viewport
- Has a blurred, semi-transparent background
- Scrolls vertically if content is long

🧩 Line-by-line explanation

```
.product-list-container { ... }
```

position: absolute;
The element is positioned relative to the nearest positioned ancestor (or the viewport).
This allows you to move it freely using top, left, etc.

top: 100vh;
This is the key.

- 100vh = 100% of the viewport height
- So the top of the element is placed **exactly one full screen below the top of the page**

Meaning:
👉 The entire panel starts **off-screen**, below the fold.

left: 0; width: 100%; height: 100vh;
This makes the panel:

- Full width
- Full height
- Positioned at the left edge

So it covers the entire screen when visible.

background-color: rgba(255, 255, 255, 0.9);
A white background with 90% opacity.
This gives a "frosted overlay" feel.

backdrop-filter: blur(5px);
This blurs whatever is *behind* the panel.
It creates that modern "glass" effect you see in iOS and macOS.

overflow-y: auto;
If the product list is long, the panel scrolls vertically.

overflow-x: hidden;
Prevents horizontal scrolling — keeps the UI clean.

transition: top 0.5s ease;
This is what makes the animation smooth.
When top changes (from 100vh → 0), it animates over 0.5 seconds.
z-index: 2;
Ensures the panel sits **above** your landing page content.

The “visible” class

```
.product-list-container.visible {
  top: 0;
}
```

When visible is added:

- `top` goes from `100vh -> 0`
- The transition animates it
- The panel slides up into view

BEFORE .visible:

```
[screen]
 ____________________
|                    |
|   Landing Page     |
|                    |
 ____________________
          [panel below screen]
```

AFTER `.visible`:

```
[screen]
 ____________________
|  Product List Panel |
|  (slides up)        |
 ____________________
```

ProductList.jsx

This is where the user gets to select plants and add to the Cart. I see several issues being flagged in VSC.

```
3   import CartItem from './CartItem';
4   function ProductList({ onHomeClick }) {
5       const [showCart, setShowCart] = useState(false);
```

I see the error message: "'onHomeClick' is missing in props validationeslintreact/prop-types". There appears to be a PropType rule from ESLint not being followed.

Solution: Add PropTypes at the bottom of the file:

- First import PropTypes

```
import PropTypes from 'prop-types';
```

- Add ProductList.propTypes after the component definition

```
ProductList.propTypes = {
    onHomeClick: PropTypes.func.isRequired,
};

export default ProductList;
```

The error disappears.

Note: The other errors/warnings are associated with the variable not being used yet.

When you click on “Get Started” it brings the <ProductList> component into view but displays no products! This will be our next task.

Product Information

The section on the page where the product data would be shown is here:

```
            {!showCart ? (
                <div className="product-grid">
                    {/* Loop through products and display them */}


                </div>
            ) : (
                <CartItem onContinueShopping={handleContinueShopping} />
            )}
```

We will discuss this in the next task.

There are two useState() hooks:

```
    const [showCart, setShowCart] = useState(false);
    const [showPlants, setShowPlants] = useState(false); // State to control the
visibility of the About Us page
```

The first one toggles between showing the cart page (similar to clicking on cart). In the Nav bar:

We can:

- Click on the Home to return to the landing page
    - Triggers the function handleHomeClick(), which executes the function prop provided in App.js

```
const handleHomeClick = (e) => {
    e.preventDefault();
    onHomeClick();
};
```

If you recall the onHomeClick invokes the App.jsx function that takes the <ProductList> component off-screen and returns to the AboutUs information.

```
const handleHomeClick = () => {
  setShowProductList(false);
};
```

Note: There are comments in the code referring to an “About Us” page. There is no such page. I think it is referring to the “landing page”

- Plants link to display the Plants information
    - This makes sense to move from Cart to Plant Product List
- Cart to display the Cart data

Task 4: Show Plant Products

The first major piece missing is viewing any plant information. The goal of this task will be to correctly display the plants in at least two categories.

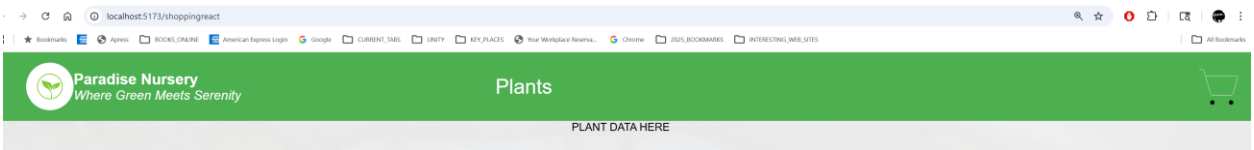
Subtask 4A: Hard code some plant information to get the card appearing correctly.

We know that the ProductList.css holds all the CSS information required to display a card. Currently we are working in the div container:

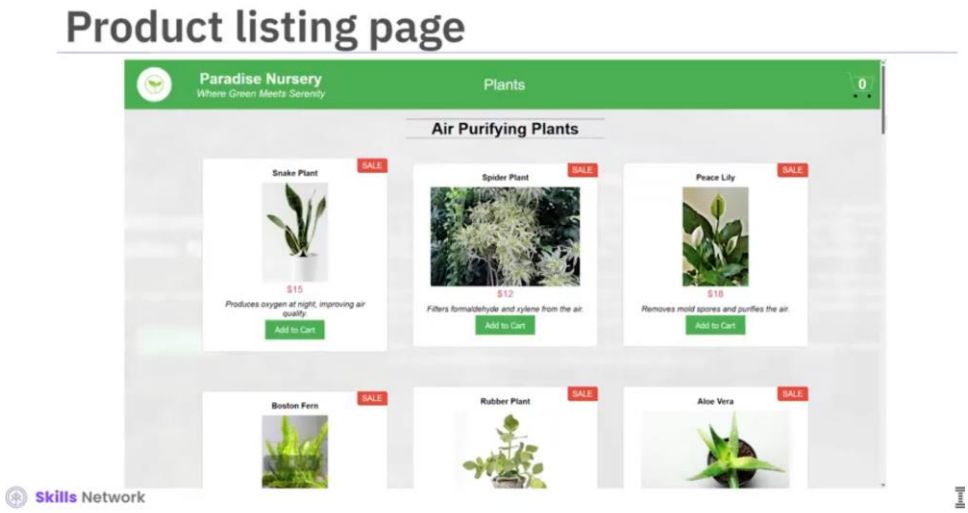
I will just print out some message to test that this is the right area:

```
            {!showCart ? (
                <div className="product-grid">
                    {/* Loop through products and display them */}
                    PLANT DATA HERE

                </div>
            )
```

Result:



This is what it should look like:



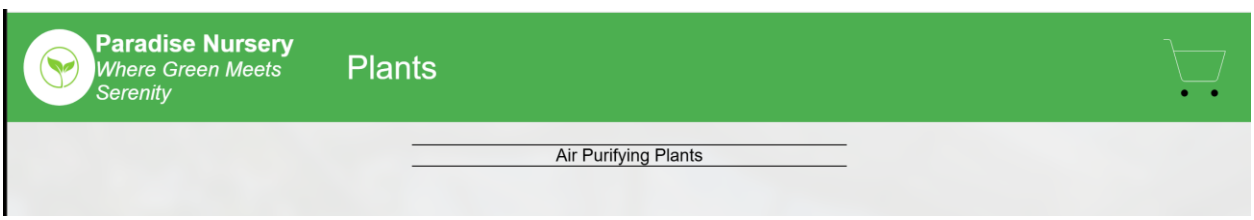
Let's hard code the plant category heading of "Air Purifying Plants". I see two CSS rules:

```
.plantname_heading{
   display: flex;
   align-items: center;
   justify-content: center;
```

```
    /* background-color: yellow; */
}
.plant_heading{
    width: 400px;
    text-align: center;
    margin: 20px;
    border: 1px solid rgb(5, 4, 4);
    border-left: none;
    border-right: none;


}
```

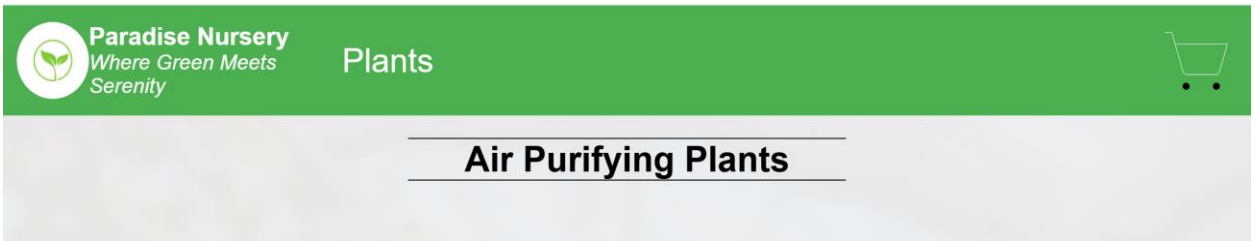
I tested both of the above and plant_heading appears to be the right one:

```
                    <div className="plant_heading">
                        Air Purifying Plants
                    </div>
```



It looks a bit light…let's add H1

```
                    <div className="plant_heading">
                        <h1>Air Purifying Plants</h1>
                    </div>
```



It looks good.

I will hard code for now one plant card with the details below:

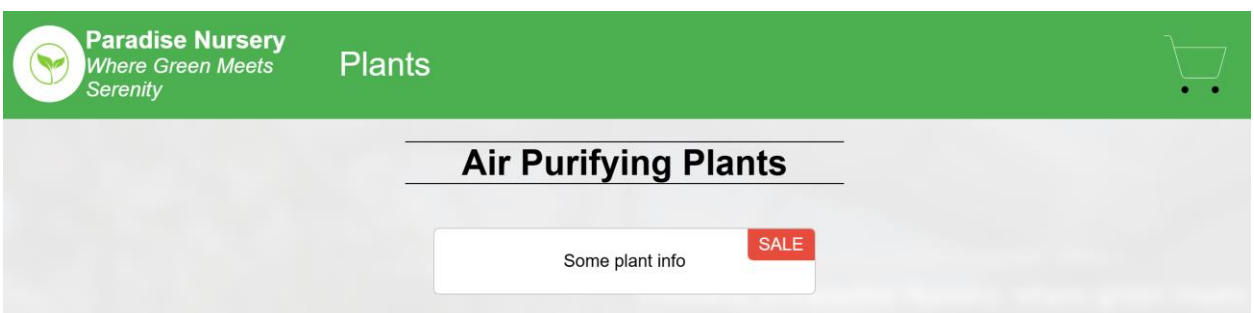
```
            category: "Air Purifying Plants",
            plants: [
```

```
                {
                    name: "Snake Plant",
                    image: "https://cdn.pixabay.com/photo/2021/01/22/06/04/snake-
plant-5939187_1280.jpg",
                    description: "Produces oxygen at night, improving air
quality.",
                    cost: "$15"
                },
```

Again, examining the CSS provided I imagine I need `product-list` to hold the cards and `product-card` for each plant information.

TEST:

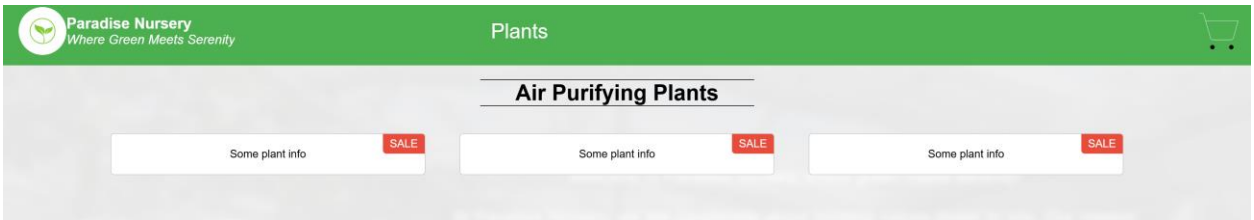
```
                <div className="product-grid">
                    {/* Loop through products and display them */}
                    <div className="plant_heading">
                        <h1>Air Purifying Plants</h1>
                    </div>
                    <div className="product-list">
                        <div className="product-card">
                            Some plant info
                        </div>
                    </div>
                </div>
```



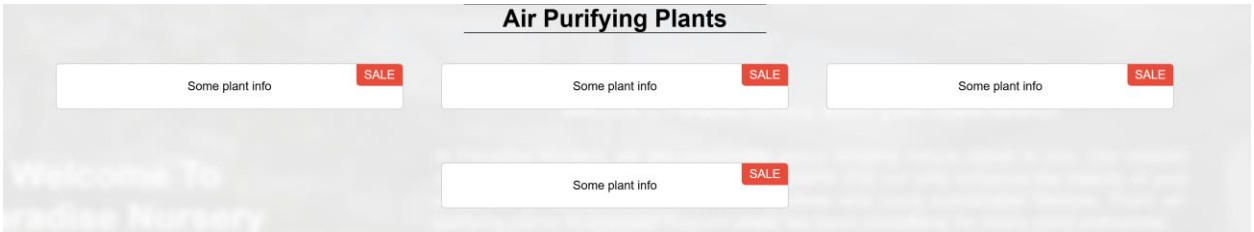
Let me see what happens when I repeat this 3 times.

```
                <div className="product-grid">
                    {/* Loop through products and display them */}
                    <div className="plant_heading">
                        <h1>Air Purifying Plants</h1>
                    </div>
                    <div className="product-list">
                        <div className="product-card">
```

```
                        Some plant info
                    </div>
                    <div className="product-card">
                        Some plant info
                    </div>
                    <div className="product-card">
                        Some plant info
                    </div>

                </div>
            </div>
```



I tested 4 times to make sure it wraps. Everything works fine:



Now let's fill in each one with the same plant data so I can make sure I am working with the right CSS rules to build a plant card.

We will use the following CSS rules:

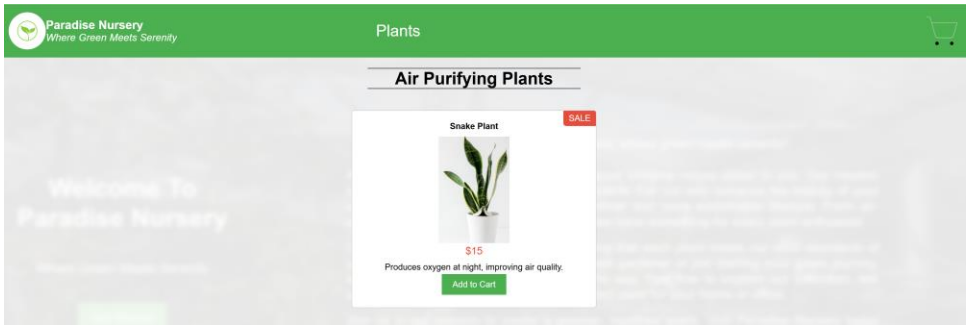
```
.product-title {
    font-weight: bold;
    margin-bottom: 10px;
}

.product-price {
    color: #e74c3c;
    font-size: 1.2rem;
    margin-bottom: 10px;
}

.product-image {
    max-width: 100%;
```

```
    height: 200px; /* Adjust height for better consistency */
}

.product-button {
    background-color: #e74c3c;
    color: #fff;
    border: none;
    padding: 10px 20px;
    cursor: pointer;
    transition: background-color 0.3s ease-in-out;
    margin-top: 10px;
}
```

TEST: Filling out the card

```
                    <div className="product-list">
                         <div className="product-card">
                            <div className="product-title">Snake Plant</div>
                            <img
src="https://cdn.pixabay.com/photo/2021/01/22/06/04/snake-plant-5939187_1280.jpg"
alt="Snake Plant" className="product-image" />
                             <div className="product-price">$15</div>
                            <p className="product-description">Produces oxygen at
night, improving air quality.</p>
                            <button className="product-button">Add to
Cart</button>
                         </div>

                      </div>
```

I see:

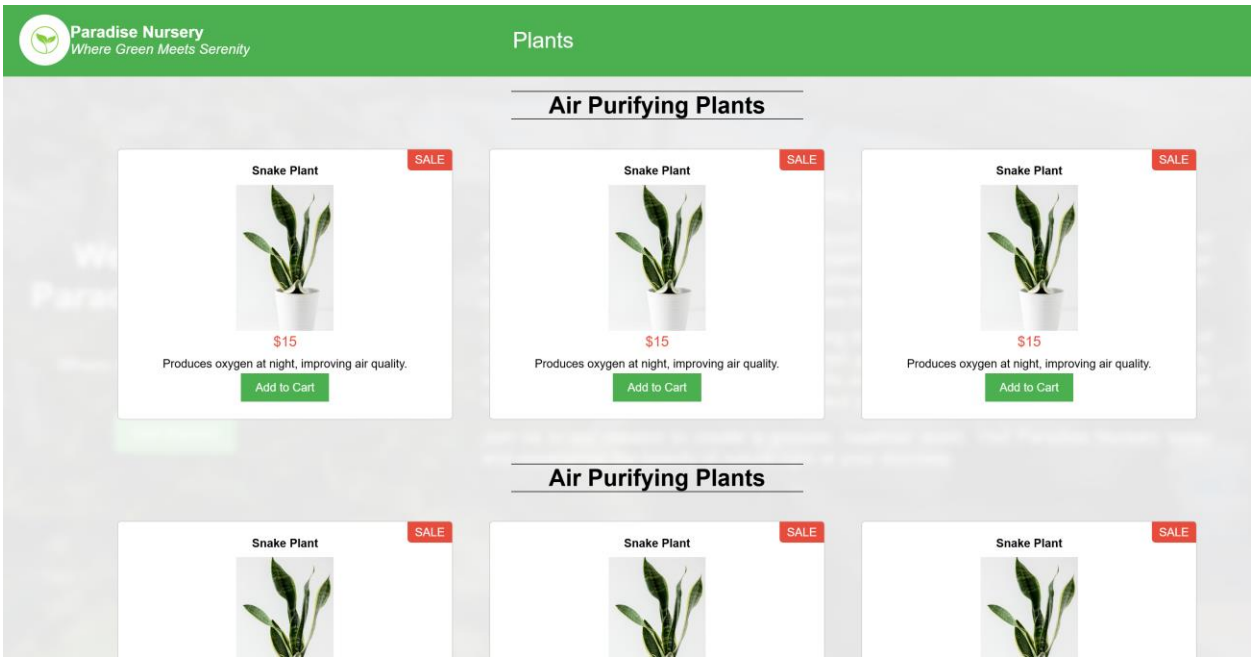


Test: Repeat the above for three plants

I think I have the template for each card and category.

TEST:

I repeated the same set (just hardcoding for now the same data):



Subtask 4B: Extracting the template:

I would like to create a new file to display a PlantCategory maybe call it DisplayPlantCategory, where I will send two props: category_name, plant_list. The first thing I will need is a function in ProductList.jsx that given a category_name returns the plants array associated with that category. Why?

The requirements stated several times that we should display at least two categories of plants. I plan on processing only two:

```
    // Extracting category names for display
    const plant_category_names = ["Air Purifying Plants", "Aromatic Fragrant
Plants"];
```

I will use the above array to know which categories we are processing for this project.

I will use the following template:

For each category in my plant_category_names array. I will obtain the plant list and generate the code:

```
                    <div className="plant_heading">
                        <h1>[CATEGORY_NAME_HERE]</h1>
                    </div>
                    <div className="product-list">
                         [FOR_EACH_PLANT_IN_CATEGORY]</
                        <div className="product-card">
                           <div className="product-title">[PLANT_NAME]</div>
                           <img src">[PLANT_IMAGE_URL]</alt="Snake Plant"
className="product-image" />
                            <div className="product-price">[PLANT_PRICE]</</div>
                           <p className="product-
description">">[PLANT_DESC]</</p>
                           <button className="product-button">Add to
Cart</button>
                        </div>
```

The above fits the information we have for each plant category:

```
                {
                    name: "Snake Plant",
                    image: "https://cdn.pixabay.com/photo/2021/01/22/06/04/snake-
plant-5939187_1280.jpg",
                    description: "Produces oxygen at night, improving air
quality.",
                    cost: "$15"
                },
```

- Created a function that when provided with a category_name returns the plant_array associated with it:

```
    // Functions to obtain plant category array
```

```
    const getPlantsByCategory = (category) => {
        const categoryData = plantsArray.find((item) => item.category ===
category);
        return categoryData ? categoryData.plants : [];
    }
```

Note: I may change things to just print all the information provided but for now I will limit myself to two plant categories.

- Add code to print the two categories:

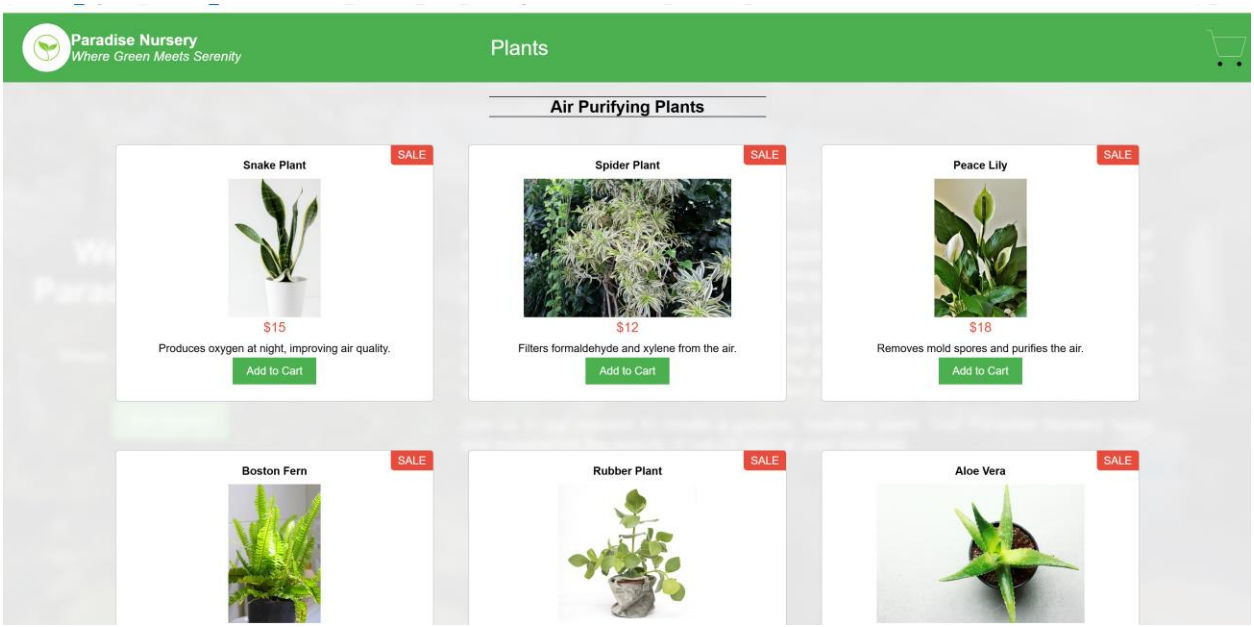
```
                <div className="product-grid">
                    {/* Loop through plant_category_names and print the plant
categories */}
                    {plant_category_names.map((category) => (
                        <div key={category}>
                            <h2 className="plant_heading">{category}</h2>
                            <div className="product-list">
                                {getPlantsByCategory(category).map((plant) => (
                                    <div key={plant.name} className="product-
card">
                                        <div className="product-
title">{plant.name}</div>
                                        <img src={plant.image} alt={plant.name}
className="product-image" />
                                        <div className="product-
price">{plant.cost}</div>
                                        <p className="product-
description">{plant.description}</p>
                                        <button className="product-button">Add to
Cart</button>
                                    </div>
                                ))}
                            </div>
                        </div>
                    ))}
                </div>
```

Note: the category heading did not appear centered as I planned. I changed the CSS rule in ProductList.css to:

```
.plant_heading{
```

```
    width: 400px;
    text-align: center;
    margin: 20px auto;
    border: 1px solid rgb(5, 4, 4);
    border-left: none;
    border-right: none;


}
```

I added “margin: 20px auto;” in order to make sure the category heading is centered.



TBD: Move this entire section into it’s own Component!

TBD: Move the plantsArray in its own file.

Task 5: Work on CartSlice.jsx

One thing bothering me and I hope to fix by the end of this task is the cart not showing the number of items on it (should be 0 by default).

Actually, I can figure out the number of items and display it now with 0 work.

Task 5A: Displaying the items count on the nav bar.

Currently, the cart is not showing any cart item count #:

We need to get the items in CartSlice state and then take the length of the # of items in the Cart.

- Import the required hooks from Redux:

```
import { useSelector, useDispatch } from 'react-redux';
```

I will only utilize useSelector for now but will initialize dispatch. Added the code:

```
function ProductList({ onHomeClick }) {
    const [showCart, setShowCart] = useState(false);
    const [showPlants, setShowPlants] = useState(false); // State to control the
visibility of the About Us page

    const dispatch = useDispatch();
    const cart = useSelector(state => state.cart.items);
```

Now to use cart.length somewhere!?

I know the place showing the cart is here:

```
                    <div>
                        <a href="#" onClick={(e) => handleCartClick(e)}
style={styleA}>
                            <h1 className='cart'>
                                <svg xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 256 256" id="IconChangeColor" height="68" width="68">
                                    <rect width="156" height="156" fill="none">
                                    </rect>
                                    <circle cx="80" cy="216" r="12"></circle>
                                    <circle cx="184" cy="216" r="12"></circle>
                                    <path
                                        d="M42.3,72H221.7l-
26.4,92.4A15.9,15.9,0,0,1,179.9,176H84.1a15.9,15.9,0,0,1-15.4-
11.6L32.5,37.8A8,8,0,0,0,24.8,32H8"
                                        fill="none"
                                        stroke="#faf9f9"
                                        stroke-linecap="round"
                                        stroke-linejoin="round"
                                        stroke-width="2"
                                        id="mainIconPathAttribute">
                                    </path>
                                </svg>
                            </h1>
```

```
                    </a>
                </div>
```

To be honest I don't know much other than guessing that <svg> is responsible for drawing the cart image we see:

The <svg> supports adding text. So I know that cart.length will provide the information I want to display:

```
                <svg
                  xmlns="http://www.w3.org/2000/svg"
                  viewBox="0 0 256 256"
                  id="IconChangeColor"
                  height="68"
                  width="68"
                >
                  <rect width="156" height="156" fill="none"></rect>
                  <circle cx="80" cy="216" r="12"></circle>
                  <circle cx="184" cy="216" r="12"></circle>
                  <path
                    d="M42.3,72H221.7l-
26.4,92.4A15.9,15.9,0,0,1,179.9,176H84.1a15.9,15.9,0,0,1-15.4-
11.6L32.5,37.8A8,8,0,0,0,24.8,32H8"
                    fill="none"
                    stroke="#faf9f9"
                    stroke-linecap="round"
                    stroke-linejoin="round"
                    stroke-width="2"
                    id="mainIconPathAttribute"
                  ></path>
                  {/* Display the number of items in the cart on top of the cart
icon */}
                  <text
                    x="128"
                    y="140"
```

```
                    textAnchor="middle"
                    fontSize="60"
                    fill="white"
                    fontWeight="bold"
                  >
                    {cart.length}
                  </text>
                </svg>
```

The <text> element inside <svg> is used to display the number of items in the cart:

Checking what a two-digit number looks..hard coding a value:

It looks OK. I think the textAnchor="middle" is responsible for it looking ok.

Task 5B: Work on adding a plant product to the Cart via "cartSlice"?

Current cartSlice:

```
import { createSlice } from '@reduxjs/toolkit';

export const CartSlice = createSlice({
  name: 'cart',
  initialState: {
    items: [], // Initialize items as an empty array
  },
  reducers: {
    addItem: (state, action) => {

    },
    removeItem: (state, action) => {
```

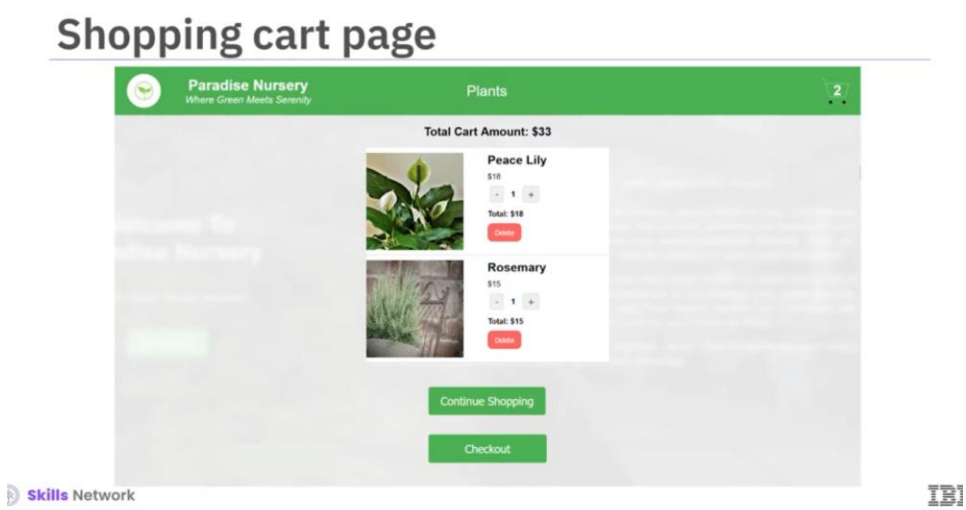
```
    },
    updateQuantity: (state, action) => {


    },
  },
});

export const { addItem, removeItem, updateQuantity } = CartSlice.actions;

export default CartSlice.reducer;
```

I first need to define what an item will look like in the CartSlice. I think it should have all the elements in the plant item with some additions.

A plant item has the following data:

```
        {
          name: "Snake Plant",
          image:
            "https://cdn.pixabay.com/photo/2021/01/22/06/04/snake-plant-
5939187_1280.jpg",
          description: "Produces oxygen at night, improving air quality.",
          cost: "$15",
        },
```

We will need `name`, `image`, and `cost` since we display those items on the shopping cart page.



We will also need `quantity`. The cart total and plant totals can be calculated dynamically.

addItem(state, action):

When the user clicks on “Add to Cart” the payload should be {name, image, cost}.

```
    addItem: (state, action) => {
      const { name, image, cost } = action.payload; // Destructure product
details from the action payload
      // Check if the item already exists in the cart by comparing names
      const existingItem = state.items.find((item) => item.name === name);
      if (existingItem) {
        // If item already exists in the cart, increase its quantity
        existingItem.quantity++;
      } else {
        // If item does not exist, add it to the cart with quantity 1
        state.items.push({ name, image, cost, quantity: 1 });
      }
    },
```

Let’s test the above by invoking addItem when the user clicks on “Add to Cart” – we should see the shopping cart count increase.

Updating the product list “Add to Cart” so it works

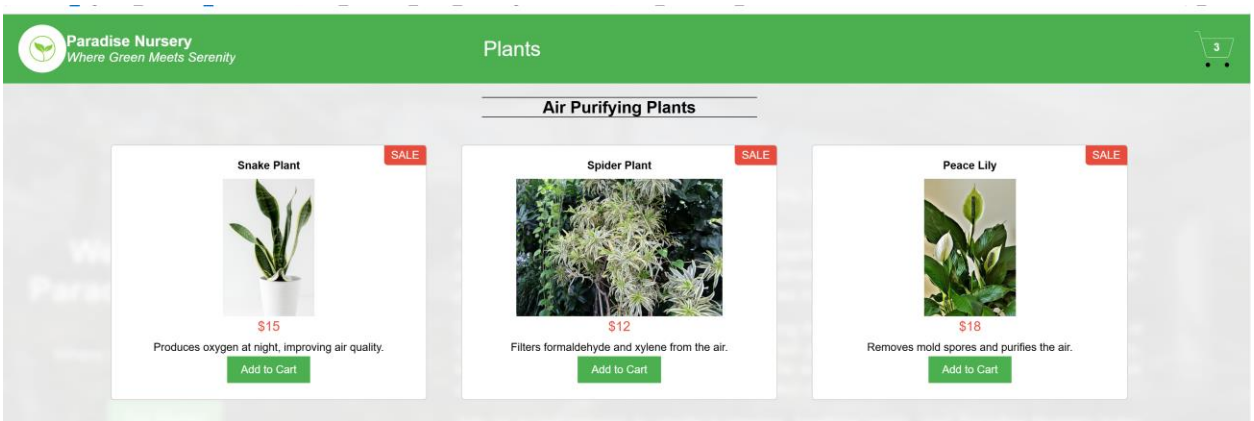
In ProductList.jsx I added the following function:

```
  // Function to add item to cart
  const handleAddToCart = (plant) => {
    dispatch(addItem(plant)); // Dispatch the addItem action with the plant
details
  };
```

The idea is that a product plant “Add to Cart” will invoke this function.

```
                    <button className="product-button" onClick={() =>
handleAddToCart(plant)}>Add to Cart</button>
```

Let’s test …the cart count should go up when I click on a plant product.



I clicked on the first three items. I need to change the button color to gray and change the text to “Added to Cart” and disable the button.

Adding the function:

```
const isPlantInCart = (plant) => {
  return cart.some((item) => item.name === plant.name);
};
```

And using the function to add the additional class rule to trigger this CSS rule:

```
.product-button.added-to-cart {
  background-color: grey; /* Grey when product is added */
}
```

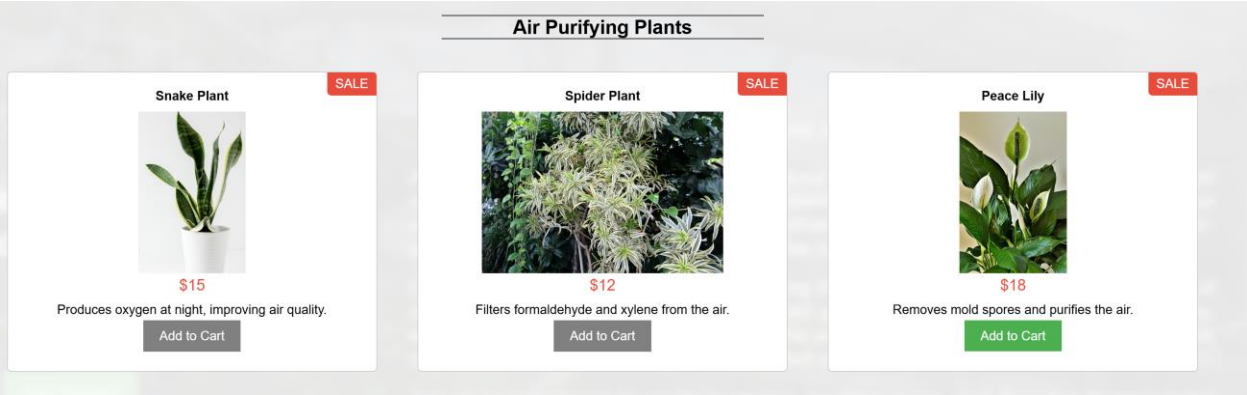
Now add code to utilize the function to:

- Update the class
- Change the button text
- Update the color from green to gray

When the user adds a plant to the cart.

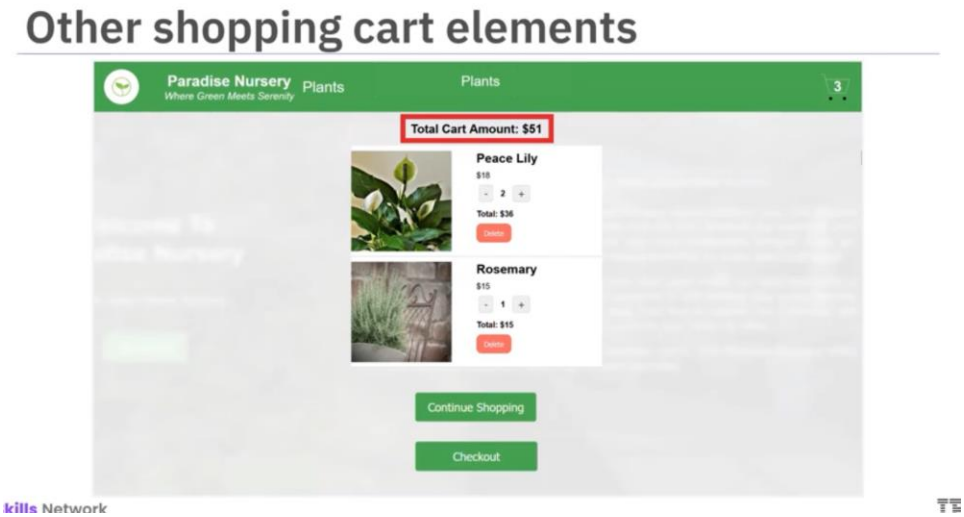
```
                    <button
                      className={`product-button ${isPlantInCart(plant) ? "added-
to-cart" : ""}`}
                      disabled={isPlantInCart(plant)} // Disable the button if
the plant is already in the cart
                      onClick={() => handleAddToCart(plant)}
                    >
```

Test the UI:



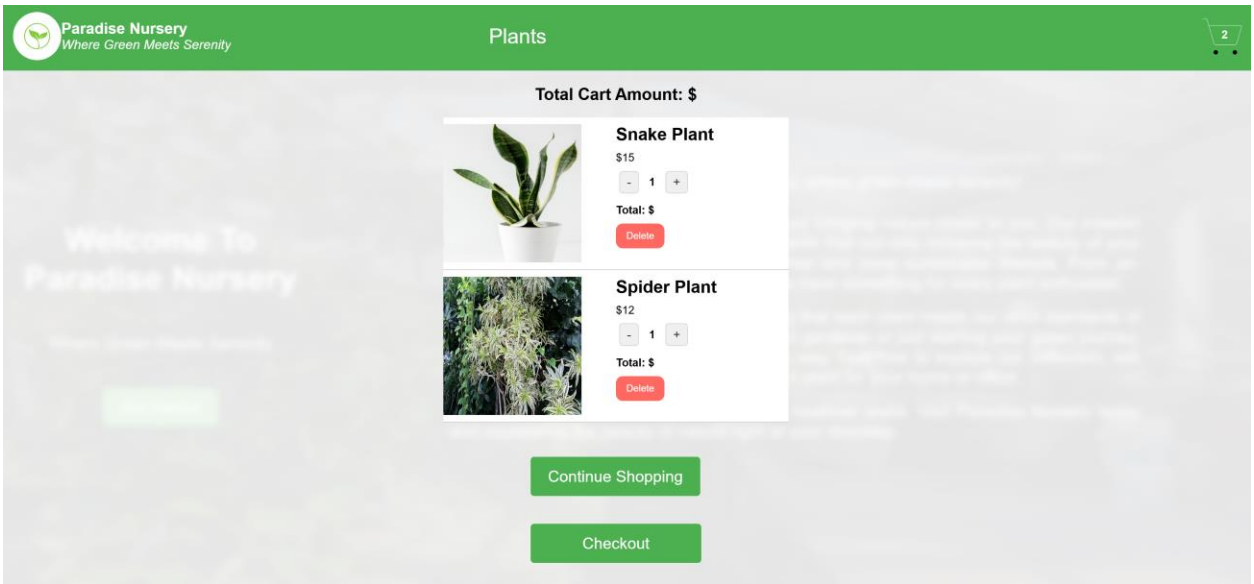
Task 5C: Work on viewing the cart?

In order to execute the other two functions in cartSlice – removeItem and updateQuantity, we need to be able to view the cart items.



Note: The calculation of “items in the cart” at the top-right corner needs to be updated to sum the quantity of all items in the cart. We will do this after we get the Cart display working and the two remaining reducer functions working.

All of the work to view the items on the cart has already been supplied as part of the starting lab files. If you add two items and click on the Cart you will see your items in the cart:



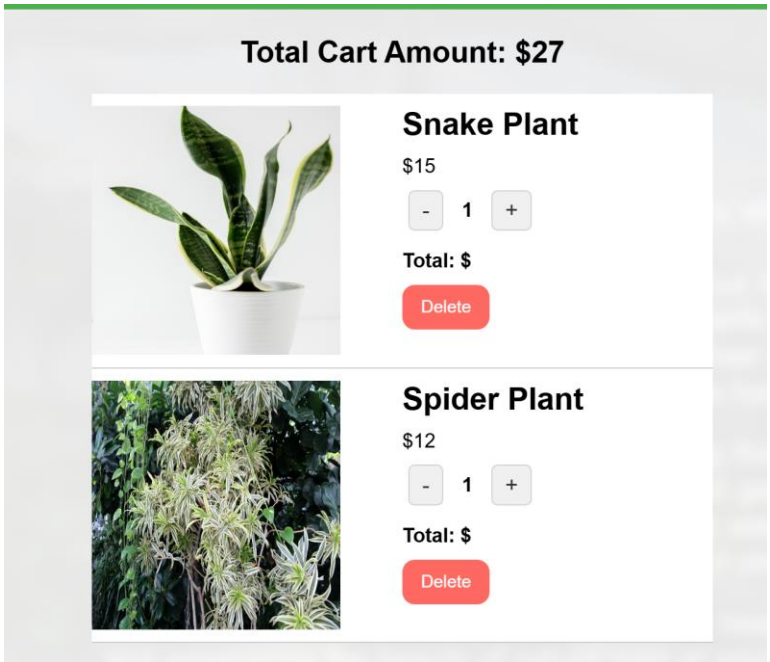
Task 5D: Fix the totals?

To calculate the cart total we just need to iterate over the items on the cart and sum (quantity * price).

```
// Calculate total amount for all products in the cart
const calculateTotalAmount = () => {
  return cart.reduce((sum, item) => {
    // Remove the "$" and convert to a number
    const price = Number(item.cost.replace("$", ""));
    return sum + item.quantity * price;
  }, 0);
};
```

The function is already invoked in the right place in the provided lab files:

```
    <h2 style={{ color: "black" }}>
      Total Cart Amount: ${calculateTotalAmount()}
    </h2>
```



- Add the individual plant totals, the code provided already calls the function:

```
              <div className="cart-item-total">
                Total: ${calculateTotalCost(item)}
              </div>
```

We just need to add code to return the value for the item.

```
  // Calculate total cost based on quantity for an item
  const calculateTotalCost = (item) => {
    const price = Number(item.cost.replace("$", ""));
    return (item.quantity * price).toFixed(2); // Return total cost with 2
decimal places
  };
```

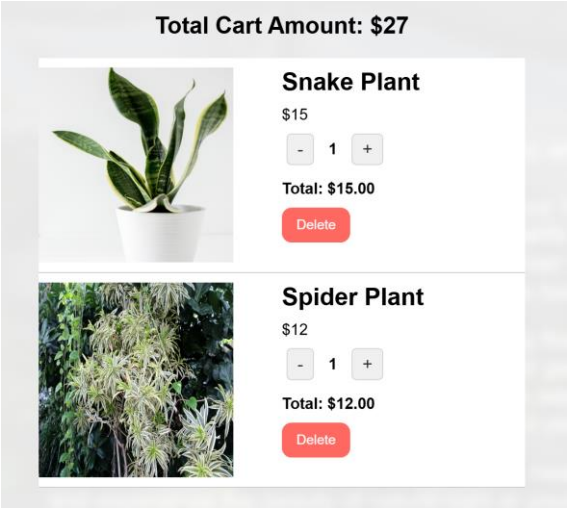
Task 5E: Finish CartSlice and invoke from the CartItem component.

- Finish removeItem in CartSlice

```
    removeItem: (state, action) => {
      state.items = state.items.filter(item => item.name !== action.payload);
    },
```

We send the plant name to the reducer function, it then filters out that plant from the list.

- Invoke removeItem when the user clicks on “Delete” button associated with a plant item.



In CartItem.jsx, we update the function:

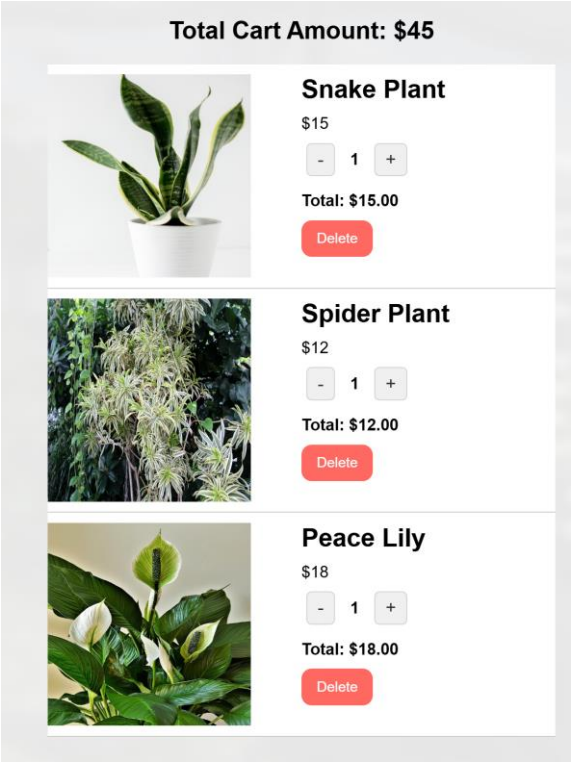
```
const handleRemove = (item) => {
  dispatch(removeItem(item.name));
};
```

Test:

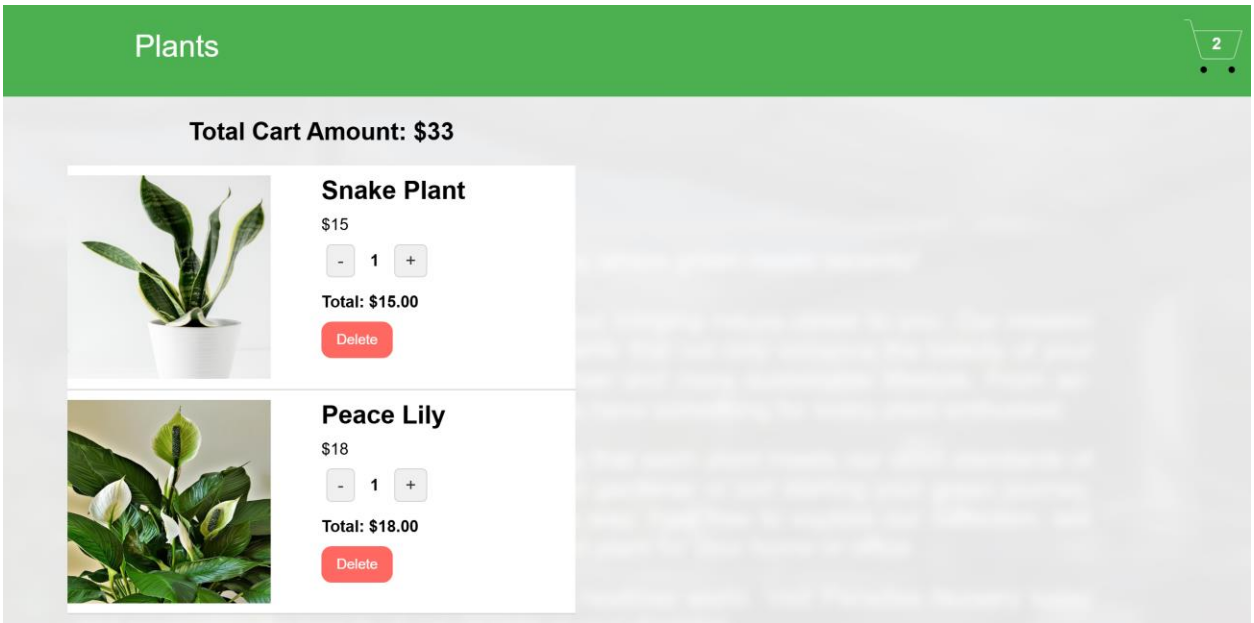
Step 1:

I added 3 items to the cart

Step 2: Click on the Cart



Step 3: Click on “Delete” button for the Spider Plant.



The plant is removed and the cart # decreases.

Step 4: Click on “Continue Shopping” but that does not work yet!

I wanted to check the the button was now enabled … let’s implement “Continue Shopping” which just means returning back to the ProductList.jsx page.

- Import PropTypes in CartItem.jsx:

```
import PropTypes from "prop-types";
```

- Add CartItem.propTypes at the bottom of the file (before export)

```
CartItem.propTypes = {
  onContinueShopping: PropTypes.func.isRequired,
};

export default CartItem;
```

- Add code to invoke onContinueShopping

```
  const handleContinueShopping = (e) => {
    console.log("Continue shopping clicked");
    e.preventDefault();
    onContinueShopping();
  };
```

FAILURE: It did not work.

TROUBLESHOOTING

I see that I needed to send the e to onContinueShopping, Fix:

```
  const handleContinueShopping = (e) => {
    console.log("Continue shopping clicked");
    e.preventDefault();
    onContinueShopping(e);
  };
```

I am watching the console log while the application runs and encountered

MORE TROUBLESHOOTING:

I see some warning messages being printed out to console that I do not understand:

```
▶Warning: Invalid DOM property `stroke-linecap`. Did you mean `strokeLinecap`? Error Component Stack
    at path (<anonymous>)
    at svg (<anonymous>)
    at h1 (<anonymous>)
    at a (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at ProductList (ProductList.jsx:8:24)
    at div (<anonymous>)
    at div (<anonymous>)
    at App (App.jsx:9:49)
  Show ignore-listed frames
```

```
▶Warning: Invalid DOM property `stroke-linejoin`. Did you mean `strokeLinejoin`? Error Component Stack
    at path (<anonymous>)
    at svg (<anonymous>)
    at h1 (<anonymous>)
    at a (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at ProductList (ProductList.jsx:8:24)
    at div (<anonymous>)
    at div (<anonymous>)
    at App (App.jsx:9:49)
  Show ignore-listed frames
```

```
▶Warning: Invalid DOM property `stroke-width`. Did you mean `strokeWidth`? Error Component Stack
    at path (<anonymous>)
    at svg (<anonymous>)
    at h1 (<anonymous>)
    at a (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at div (<anonymous>)
    at ProductList (ProductList.jsx:8:24)
    at div (<anonymous>)
    at div (<anonymous>)
    at App (App.jsx:9:49)
  Show ignore-listed frames
```

- First I will remove the console.log() message I added to the CartItems function: handleContinueShopping.
- Exploring the source for the Warning messages above:

React is giving you that warning because **SVG attributes use kebab-case in plain HTML**, but **React requires camelCase for all DOM properties**, including SVG ones.

So React sees:

```
Html

stroke-linecap
stroke-linejoin
stroke-width
```

…and says:

"Those aren't valid React DOM properties. Did you mean `strokeLinecap`, `strokeLinejoin`, `strokeWidth`?"

React is right — it's just enforcing its own naming rules.

React maps JSX attributes to JavaScript object properties.
JavaScript object keys can't use hyphens without quotes, so React standardizes everything to camelCase.

Examples:

SVG Attribute	React JSX Equivalent
stroke-linecap	strokeLinecap
stroke-linejoin	strokeLinejoin
stroke-width	strokeWidth

How to fix the SVG:

Just change the attributes to camelCase:

```
<path
  d="M42.3,72H221.7l-26.4,92.4A15.9,15.9,0,0,1,179.9,176H84.1a15.9,15.9,0,0,1-15.4-11.6L32.5,37.8A8,8,0,0,0,24.8,32H8"
  fill="none"
  stroke="#faf9f9"
  strokeLinecap="round"
  strokeLinejoin="round"
  strokeWidth="2"
  id="mainIconPathAttribute"
/>
```

I changed in ProductList.jsx and everything still works okay, and the Warnings disappeared.

Completed code in CartSlice.jsx:

```
import { createSlice } from "@reduxjs/toolkit";

export const CartSlice = createSlice({
  name: "cart",
  initialState: {
    items: [], // Initialize items as an empty array
  },
  reducers: {
    addItem: (state, action) => {
```

```
      const { name, image, cost } = action.payload; // Destructure product
details from the action payload
      // Check if the item already exists in the cart by comparing names
      const existingItem = state.items.find((item) => item.name === name);
      if (existingItem) {
        // If item already exists in the cart, increase its quantity
        existingItem.quantity++;
      } else {
        // If item does not exist, add it to the cart with quantity 1
        state.items.push({ name, image, cost, quantity: 1 });
      }
    },
    removeItem: (state, action) => {
      state.items = state.items.filter(item => item.name !== action.payload);
    },
    updateQuantity: (state, action) => {
      const { name, quantity } = action.payload;
      const existingItem = state.items.find((item) => item.name === name);
      if (existingItem) {
        existingItem.quantity = quantity;
      }
    },
  },
});

export const { addItem, removeItem, updateQuantity } = CartSlice.actions;

export default CartSlice.reducer;
```

Completing the `handleIncrement` and `handleDecrement` function in `CartItem`:

```
const handleIncrement = (item) => {
    dispatch(updateQuantity({ name: item.name, quantity: item.quantity + 1 }));
  };

  const handleDecrement = (item) => {
    if (item.quantity > 1) {
      dispatch(updateQuantity({ name: item.name, quantity: item.quantity - 1 }));
    } else {
      // If quantity is 1 and user clicks decrement, remove the item from the
cart
      dispatch(removeItem(item.name));
    }
```

```
  };
```

Note: An item is removed from the Cart when the quantity count becomes 0.

Test:

Add items to the Cart. Test incrementing and decrementing values. Make sure all “totals” and “counts” are consistent.

Task #6: Fix the cart total

The Cart total displays the count on the number of different plants you have on the Cart. I need to sum quantity for each item on the cart.

In ProductList.jsx I added a new function:

```
  const getTotalCartQuantity = () => {
    return cart.reduce((sum, item) => sum + item.quantity, 0);
  };
```

I replaced the code in the svg text area to:

```
                  <text
                    x="128"
                    y="140"
                    textAnchor="middle"
                    fontSize="60"
                    fill="white"
                    fontWeight="bold"
                  >
                    {getTotalCartQuantity()}
                  </text>
```

Task #7: Implement Checkout Button

This will just display a message like “Coming Soon.”

Added the code:

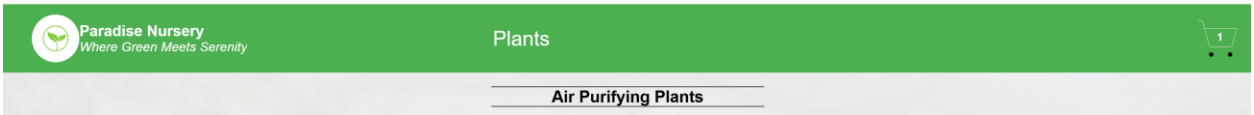
```
        <button className="get-started-button1" onClick={() => alert("Coming
Soon. Checkout feature is under development.")}>Checkout</button>
```

To CartItem.jsx

Task #8: Minor UI Improvement

I would like to make some minor improvements to the UI

Task #8A – Center the Plants link at the top



The Plants link on the Nav bar looks off center. I would like to figure out how to center it, regardless of screen/monitor size.

ISSUE #1: There appears to be a typo in code that was provided…checking the original…

```
    const styleObj = {
        backgroundColor: '#4CAF50',
        color: '#fff!important',
        padding: '15px',
        display: 'flex',
        justifyContent: 'space-between',
        alignIems: 'center',
        fontSize: '20px',
    }
```

The above is the original and there appears to be a typo in the highlighted rule. It should be alignItems!!

Updated code:

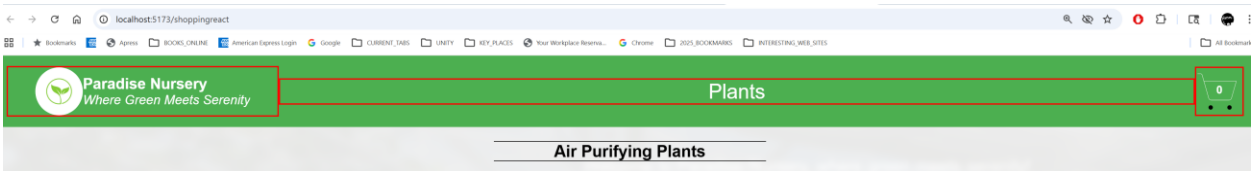
```
const styleObj = {
  backgroundColor: "#4CAF50",
  color: "#fff!important",
  padding: "15px",
  display: "flex",
  justifyContent: "space-between",
  alignItems: "center",
  fontSize: "20px",
};
```

Issue #2: I don't think the tags are correct. I need it to form:

LEFT_BLOCK|MIDDLE_BLOCK|RIGHT_BLOCK

Where the LEFT_BLOCK is the Logo (which needs a bit of space between the logo image and the text, that is the next Task). The MIDDLE_BLOCK the Plants link and the RIGHT_BLOCK the cart. It appears the MIDDLE_BLOCK and RIGHT_BLOCK are under the same div…let me separate.

I partitioned the tags defining the navbar into the blocks defined above. I added borders to each element to see what was going on. I finally got this configuration:



I centered the Plants not so much on the screen but within between the LEFT_BLOCK and MIDDLE_BLOCK.

```
const styleObjUl = {
  display: "flex",
  justifyContent: "center",
  alignItems: "center",
  flex: 1,
};
```

Unfortunately, the Plants is still not centered because the LEFT_BLOCK and RIGHT_BLOCK are different sizes. So I do I:

```
[ LEFT_BLOCK (same width as RIGHT_BLOCK)] [ MIDDLE_BLOCK ] [RIGHT_BLOCK (same width as LEFT_BLOCK)]
```

- LEFT_BLOCK is wide (logo + text)
- RIGHT_BLOCK is narrow (cart icon)
- MIDDLE_BLOCK gets pushed right because LEFT_BLOCK > RIGHT_BLOCK

I need to make LEFT_BLOCK and RIGHT_BLOCK equal-width flex items.

Working with Copilot I managed to figure out:

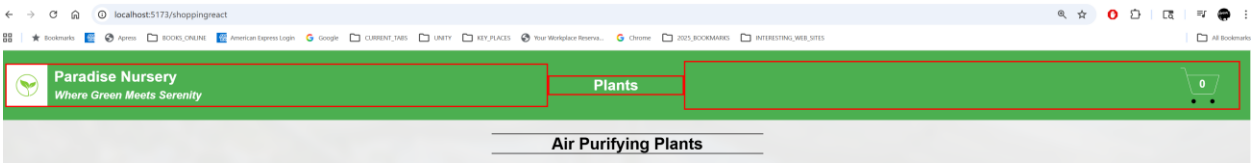
- Remove the style code in `ProductList.jsx`:

```
const styleObjUl = {
  display: "flex",
  justifyContent: "center",
  alignItems: "center",
  flex: 1,
};
```

```
// Create style rule for right side cart icon
const styleObjCart = {
  display: "flex",
  justifyContent: "center",
  alignItems: "center"
};

const styleA = {
  color: "white",
  fontSize: "30px",
  textDecoration: "none",
};
```

It is bad coding to have a mix inline CSS rules with CSS rules defined in ProductList.css! Quite frankly it feels like more than one person had a hand in styling. In any case, I removed the above code and put all the rules for styling in CSS.

I got it so that it is now appearing as:



I will remove my debugging border code. My final set of ProductList.css is:

```
/* Reset some default styles */
body, h1, ul {
    margin: 0;
    padding: 0;
}

/* Set a background color */
body {
    font-family: Arial, sans-serif;
    background-color: #f0f0f0;
}

/* Navbar */
/* NAVBAR CONTAINER */
.navbar {
  background-color: #4CAF50;
  color: white;
  padding: 15px;
```

```
  display: flex;
  align-items: center;
  justify-content: space-between;
}

/* LEFT and RIGHT get equal width */
.nav-left,
.nav-right {
  flex: 1;
  display: flex;
  align-items: center;
}
.nav-right {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: flex-end;   /* ☆ pushes cart to the far right */
  padding-right: 20px;         /* ☆ adds spacing from the edge */
}
/* CENTER stays centered and does NOT grow */
.nav-center {
  flex: 0;
  min-width: 200px; /* or whatever you want */
  display: flex;
  justify-content: center;
}
.nav-center a {
  font-size: 22px;
}

.nav-left h3 {
  font-size: 22px;
  margin: 0;
}

.nav-left i {
  font-size: 16px;
}
/* Logo + text */
.luxury {
  display: flex;
  align-items: center;
  gap: 10px;
}
```

```
.luxury img {
  height: 60px;
  width: 60px;
}

/* Links */
.navbar a {
  color: white;
  font-size: 22px;
  font-weight: bold;
  text-decoration: none;
}

/* Cart icon wrapper */
.cart-icon {
  display: flex;
  justify-content: flex-end;
  width: 100%;
}

/* DEBUG BORDERS (optional) */
/*
.nav-left { border: 2px solid blue; }
.nav-center { border: 2px solid yellow; }
.nav-right { border: 2px solid green; }
*/




/* Product Grid */
.product-grid {
    display:flex;
    flex-direction: column;
    width: 100vw;
    align-items: center;
    justify-content: center;
}
.product-list {
    display: flex;
    flex-wrap: wrap;
    gap: 50px;
    /* background-color: pink; */
    /* justify-content: space-between; */
    padding: 20px;
```

```
    width: 100%;
    align-items: center;
    justify-content: center;
}


/* Product Card */
.product-card {
    flex: 0 0 calc(33.33% - 20px); /* Adjust width for 3 cards per row with 20px
gap */
    max-width: calc(26.33% - 20px); /* Adjust max-width for 3 cards per row with
20px gap */
    margin-bottom: 20px;
    padding: 20px;
    background-color: #fff;
    border: 1px solid #ccc;
    border-radius: 5px;
    text-align: center;
    position: relative;

    gap: 20px;
}

/* Pseudo-classes - Hover effect on product button */
.product-card:hover {
    transform: scale(1.05);
    transition: transform 0.3s ease-in-out;
    z-index: 1;
}

.product-title {
    font-weight: bold;
    margin-bottom: 10px;
}

.product-price {
    color: #e74c3c;
    font-size: 1.2rem;
    margin-bottom: 10px;
}

.product-image {
    max-width: 100%;
    height: 200px; /* Adjust height for better consistency */
}
```

```
.product-button {
    background-color: #e74c3c;
    color: #fff;
    border: none;
    padding: 10px 20px;
    cursor: pointer;
    transition: background-color 0.3s ease-in-out;
    margin-top: 10px;
}

.product-button:hover {
    background-color: #c0392b;
}

/* Pseudo-elements - Sale badge */
.product-card::before {
    content: "SALE";
    background-color: #e74c3c;
    color: #fff;
    position: absolute;
    top: 0;
    right: 0;
    padding: 5px 10px;
    border-radius: 0 0 0 5px;
}
.tag_home_link{
    display: flex;
    /* background-color: red; */
    flex-direction: column;
    align-items: center;
    justify-content: center;
    margin-left: 50px;
    color: white;
    text-decoration: none;
    font-size: 20px;
}
.tag_home_link h3{
    font-size: 30px;
}
.tag a{
    text-decoration: none;
}
.tag {
    width: 400px;
```

```
    display: flex;
    align-items: center;
    justify-content: center;
    border: 2px solid blue;
}

.tag img {
    height: 70px;
    width: 70px;
    border-radius: 70%;

}


.cart{
    color: white;
    display: flex;
}
.cart_quantity_count{
    margin-top: 16px;
    /* background-color: red; */
    margin-left: 27px;
    position: absolute;
    font-size: 29px;

}
.plantname_heading{
   display: flex;
   align-items: center;
   justify-content: center;
    /* background-color: yellow; */
}
.plant_heading{
    width: 400px;
    text-align: center;
    margin: 20px auto;
    border: 1px solid rgb(5, 4, 4);
    border-left: none;
    border-right: none;


}
/* Add Media Query for responsiveness */
@media (max-width: 768px) {
    .product-card {
```

```
        flex: 1 1 calc(50% - 20px); /* Adjust width for 2 cards per row with 20px
gap on smaller screens */
        max-width: calc(50% - 20px); /* Adjust max-width for 2 cards per row with
20px gap on smaller screens */
    }
}
/* ProductList.css */

.product-button {
    background-color: #4CAF50; /* Green */
    border: none;
    color: white;
    padding: 10px 20px;
    text-align: center;
    text-decoration: none;
    display: inline-block;
    font-size: 16px;
    margin: 4px 2px;
    transition-duration: 0.4s;
    cursor: pointer;
  }

  .product-button:hover {
    background-color: #45a049;
  }

  .product-button.added-to-cart {
    background-color: grey; /* Grey when product is added */
  }



  @media (max-width: 1200px) {
    .product-card {
      flex: 1 1 calc(33.33% - 20px); /* Adjust width for 3 cards per row with
20px gap on medium screens */
      max-width: calc(33.33% - 20px); /* Adjust max-width for 3 cards per row
with 20px gap on medium screens */
    }
  }

  @media (max-width: 768px) {
    .product-card {
      flex: 1 1 calc(50% - 20px); /* Adjust width for 2 cards per row with 20px
gap on small screens */
```

```
      max-width: calc(50% - 20px); /* Adjust max-width for 2 cards per row with
20px gap on small screens */
    }
    .navbar {
        flex-direction: column; /* Change flex direction to stack items
vertically */
        align-items: center; /* Align items to the center of the container */
      }

      .tag {
        margin-bottom: 20px; /* Add margin bottom for spacing */
        text-align: center; /* Align text to the center */
      }

      .ul {
        display: flex; /* Set display to flex */
        flex-direction: column; /* Change flex direction to stack items
vertically */
        gap: 10px; /* Add gap between items */
      }

      .ul div {
        text-align: center; /* Align text to the center */
      }
  }
```

Task #9: Checklist Review

Task #9A: Show 3 categories

I only show 2 categories. I changed the category list to:

```
  // Extracting category names for display
  const plant_category_names = [
    "Air Purifying Plants",
    "Aromatic Fragrant Plants",
    "Low Maintenance Plants"
  ];
```

Task #10: Deploying the Application with GitHub Pages

Step 1: Install gh-pages

```
npm install gh-pages --save-dev
```

Step 2: Update package.json adding “predeploy” and “deploy”

```
"scripts": {
    "dev": "vite",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist",
    "build": "vite build",
    "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-
warnings 0",
    "preview": "vite build; vite preview --host"
  },
```

Step 3: Update vite.config.js to add my repo name:

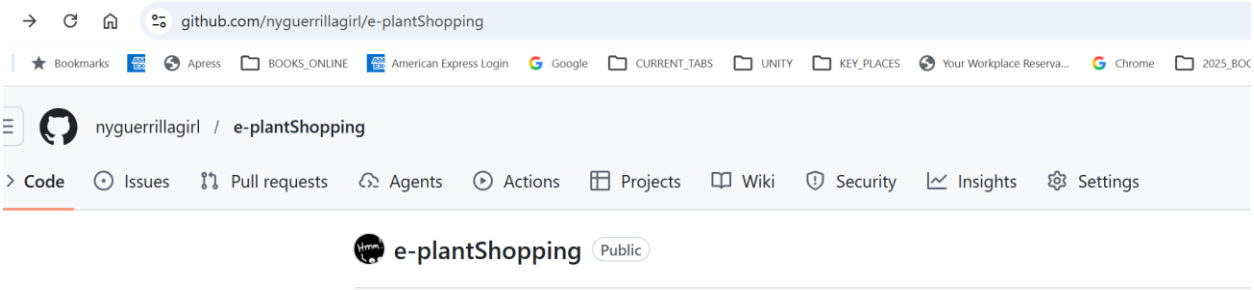
```
export default defineConfig({
  base: "/e-plantShopping/",
  plugins: [react()],
})
```

Step 4: Run the following:

```
npm run build
npm run deploy
```

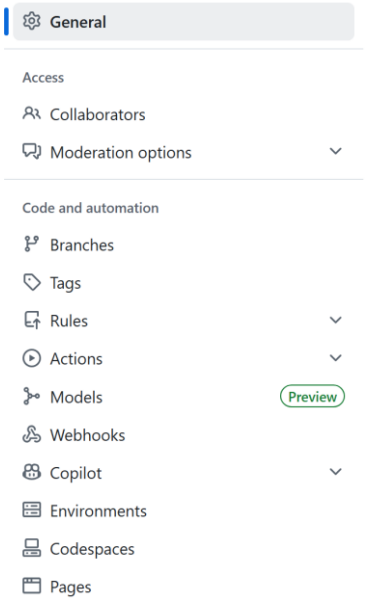
```
> npm run build


> shoppingcart@0.0.0 build
> vite build

vite v5.2.10 building for production...
✓ 57 modules transformed.
dist/index.html                   0.74 kB │ gzip:  0.42 kB
dist/assets/index-Bw8IyepC.css    6.95 kB │ gzip:  1.86 kB
dist/assets/index-BBLjO_yb.js   181.35 kB │ gzip: 59.71 kB
✓ built in 495ms

> shoppingcart@0.0.0 deploy
> gh-pages -d dist

Published
```

Step 5: Open Github repo page

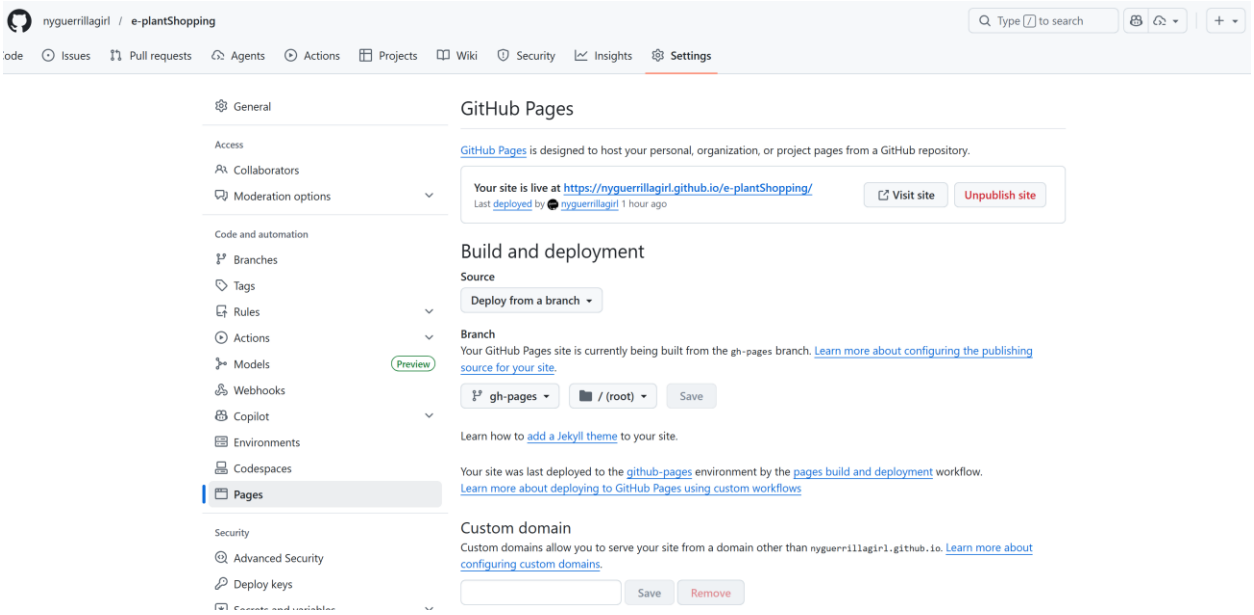


The URL is: https://github.com/nyguerrillagirl/e-plantShopping

Step 6: Click on Settings

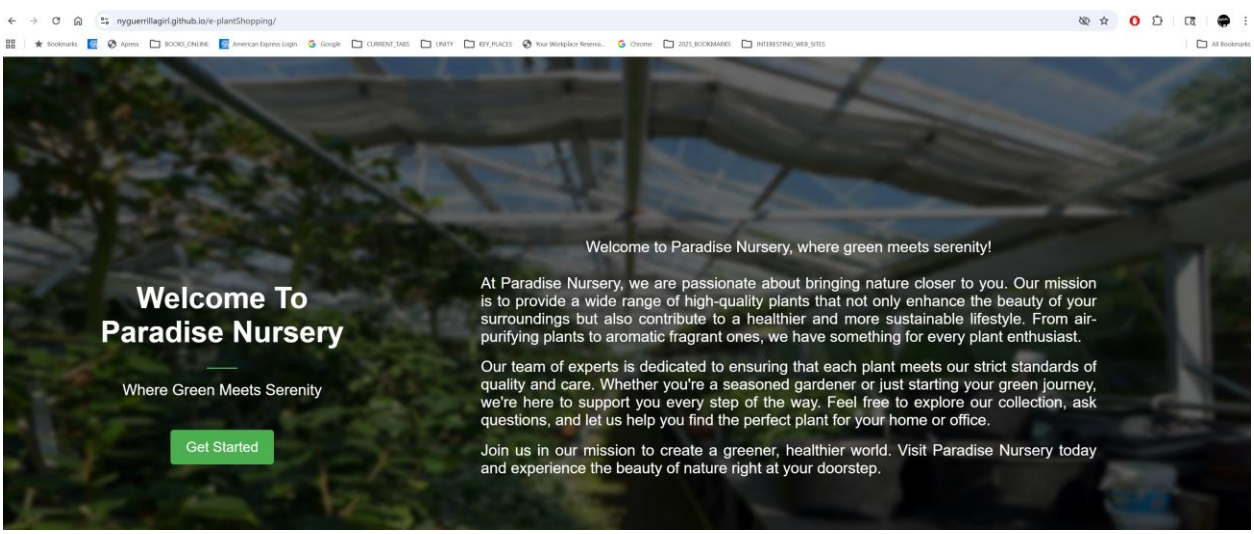


Step 7: Click on Pages



There was nothing I had to do here. Note the URL of your application: https://nyguerrillagirl.github.io/e-plantShopping/

Step 8: Click on the URL shown (I had to wait at least 5 minutes to see my application).



Play around with the application.

Task #11: I would like "Checkout" to clear the cart and leave the user on the Homepage.

Step 1: Add a new reducer function to CartSlice – clearCart(state, action).

```
    clearCart: (state) => {      state.items = [];
    }
```

Step 2: Update the export list in CartSlice:

```
export const { addItem, removeItem, updateQuantity, clearCart } =
CartSlice.actions;
```

Step 3: Update ProductList.jsx to send CartItem an additional prop to return to Home page.

```
        <CartItem onContinueShopping={handleContinueShopping}
onHomeClick={handleHomeClick}/>
```

Step 4: Update signature of CartItem props and PropTypes

```
const CartItem = ({ onContinueShopping, onHomeClick }) => {
```

Step 5: update import for new CartSlice reducer function

```
import { removeItem, updateQuantity, clearCart} from "./CartSlice";
```

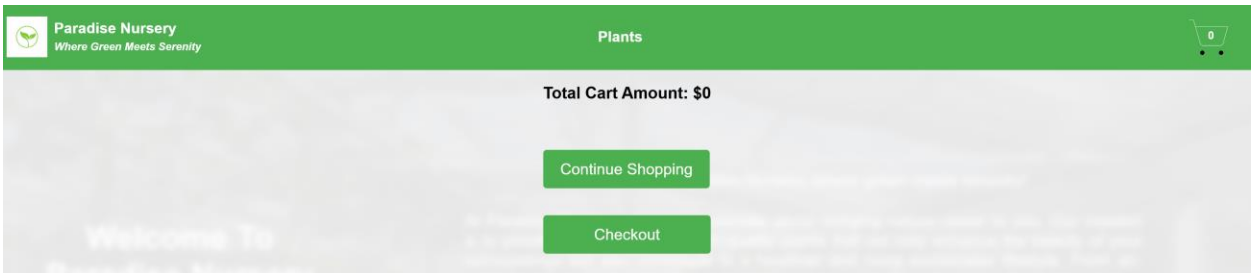
Step 6: Create new function handleCheckout to clear the cart and return user to home page:

```
  // New function named handleCheckout to invoke clearCart action and return to
home page
  const handleCheckout = (e) => {
    e.preventDefault();
    dispatch(clearCart()); // Clear the cart after checkout
    onHomeClick(); // Navigate back to the home page
  };
```

Step 7: Utilize the new function on Checkout button click:

```
        <button className="get-started-button1" onClick={(e) =>
handleCheckout(e)}>Checkout</button>
```

Issue: It is not working as planned. The cart is being emptied but I am staying on cart so I see:



Step 8: Let me create the cart cleaning and returning back home in the ProductList page.

The relationship between components is:

```
App (or parent)
└── ProductList
      ├── Product grid
      └── CartItem
```

ProductList controls which view is shown using:

```
const [showCart, setShowCart] = useState(false);
```

So:

- showCart = true → show CartItem
- showCart = false → show product list

This means **ProductList is the "router"** for your app.

That means the work needs to be done in ProductList and the function sent to CartItem.

Fix:

Add new function to ProductList:

```
const handleCheckoutReturnHome  = () => {
  setShowCart(false);
  onHomeClick();
};
```

Send to <CartItem> as prop:

```
        <CartItem onContinueShopping={handleContinueShopping}
onCheckoutReturnHome={handleCheckoutReturnHome}/>
```

Update the CartItem component signature:

```
const CartItem = ({ onContinueShopping, onCheckoutReturnHome }) => {
```

Update PropTypes:

```
CartItem.propTypes = {
  onContinueShopping: PropTypes.func.isRequired,
  onCheckoutReturnHome: PropTypes.func.isRequired,
};
```

Keep the code invoking local handleCheckout in CartItem component:

```
       <button className="get-started-button1" onClick={(e) =>
handleCheckout(e)}>Checkout</button>
```

Update handlCheckout:

```
  // New function named handleCheckout to invoke clearCart action and return to
home page
  const handleCheckout = (e) => {
    e.preventDefault();
    alert("Checkout function not implemented yet. Clearing the cart and returning
to home page.");
    dispatch(clearCart()); // Clear the cart after checkout
    onCheckoutReturnHome(); // Navigate back to the home page
  };
```

It works now as planned.

Task #12: Final Steps

- update git
- rebuild and deploy
- check at https://nyguerrillagirl.github.io/e-plantShopping/
- Upload above URL and URL of repo with final version